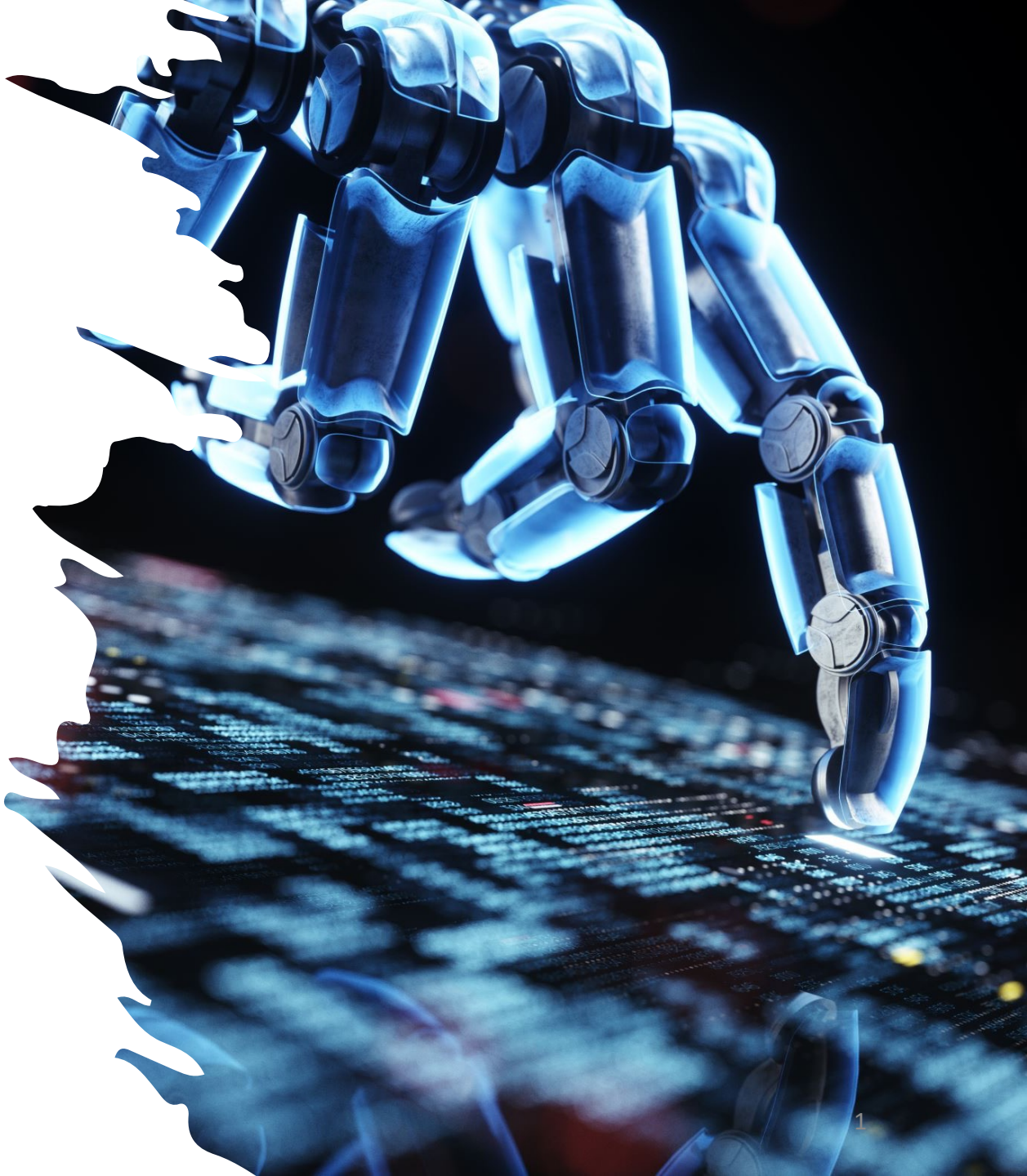
Reinforcement Learning

Teil 2

- Exploration
- Offline RL



Exploration

Schwierigkeitsgrad aus Sicht eines RL-Agenten

Eher leichte Aufgabe

Breakout

Was macht eine Aufgabe schwierig für einen RL-Agenten?

Extrem schwierige Aufgabe

Montezuma's revenge

Sichtweise eines menschlichen Spielers kann sich unterscheiden

Montezuma’s revenge

- Schlüssel aufsammeln → Reward
- Tür öffnen → Reward
- Getötet werden von Totenkopf → nichts (gut? Schlecht?)
- Problem: Das Spiel zu beenden korreliert nur schwach mit Reward-Ereignissen
- Menschliche Spieler wissen was zu tun ist, weil sie die Spielelemente verstehen (Schlüssel, Totenkopf, …)

Sicht eines RL-Algorithmen verstehen

UNO

- Sie kennen die Regeln nicht
- Können Regeln nur durch Ausprobieren herausfinden
- Regeln machen evtl. keinen Sinn aus Ihrer Sicht
- Zeitlich ausgedehnte Aufgaben wie Montezuma's revenge werden umso schwieriger
  - Je größer der Zeithorizont ist
  - Wie wenig Sie über die Regeln wissen
- Stellen Sie sich vor, Ihr Lebensziel bestünde daraus, 42 Spiele UNO zu gewinnen…
- … und Sie wüssten vorher nichts davon

Weitere Beispiele

- Das beschriebene Problem beschränkt sich nicht nur auf Spiele
- Es besteht bei sämtlichen Aufgaben eines RL-Agenten
- Beispiel aus der Robotik:
  - Gegenstände mit einer (Roboter-) Hand bewegen
  - Werkzeuge nutzen
  - Türen oder Schubladen öffnen
  - …
  - Erfordert komplexes Zusammenspiel mehrerer Finger und das Erlernen, wie auf die jeweilige Umgebung Einfluss genommen werden kann

Exploration und Exploitation

- Wie kann ein Agent ein Verhalten erlernen, das eine zweitlich ausgedehnte Sequenz komplexer Aktionen erfordert, die individuell keinen Reward bringen?
- Wie kann ein Agent sich entscheiden, ob ein neues Verhalten probiert (mit dem Ziel, höhere Rewards zu entdecken) oder weiterhin das bestbekannte Verhalten fortgesetzt werden soll
- Sind beides Sichtweisen desselben Problems
  - Exploitation: Tun, was wir aktuell für das beste halten
  - Exploration: Neue Dinge ausprobieren in der Hoffnung noch höhere Rewards zu erhalten

Exploration ist schwer

- Können wir eine **optimale** Explorationsstrategie finden?
- Was bedeutet optimal in dem Kontext?
  - Z.B. Regret im Vergleich zu einer Bayes-optimalen Strategie

Multi-armed Bandits (1-step stateless RL problems)

Contextual Bandits (1-step RL problems)

Kleine, endliche MDPs (z.B. berechenbare Planung, Modell-basiertes RL)

Große, unendliche MDPs, kontinuierliche Räume

Theoretisch lösbar

Theoretisch unlösbar

Bandits

- Was sind Banditen?

→ Einfachstes aller Explorationsprobleme

One-armed Bandit

$$\mathcal{A} = \{\text{am Arm ziehen}\}$$

$$r(a) = ?$$

Multi-armed Bandit

$$\mathcal{A} = \{\text{Arm}_1, \text{Arm}_2, \dots, \text{Arm}_n\}$$

$$r(a_i) = ?$$

Nehmen an $r(a_i) \sim p(r|a_i)$

Wie können wir Bandits definieren?

- Annahme: $r(a_i) \sim p_{\theta_i}(r_i)$,
- z.B. $p(r_i = 1) = \theta_i$ und $p(r_i = 0) = 1 - \theta_i$
- Das definiert einen POMDP mit $s = [\theta_1, \dots, \theta_n]$
- Wir haben einen "belief state" $\hat{p}(\theta_1, \dots, \theta_n)$
- Lösung des POMDP → optimale Explorationsstrategie ("Welche Aktion ist jetzt optimal, wenn ich auch berücksichtige, was ich dadurch lerne und wie mir dieses Wissen in allen künftigen Schritten hilft?")
- Aber: Das ist ein riesiges, nicht allgemein lösbares Problem.
- Gleichzeitig können wir sehr gute Ergebnisse mit einfachen Strategien erzielen
- Wie die Qualität eines Explorationsalgorithmus messen?
- →Regret: Differenz zur optimalen Policy zum Zeitpunkt T:

Tatsächlich durchgeführte Aktion

$$Reg(T) = TE[r(a^*)] - \sum_{t=1}^{T} r(a_t)$$

Erwartungswert beste Aktion

Drei Explorationsstrategie-Klassen

Optimale Explorationsstrategien für einfache MDPs

Bandit-Probelme lösen

$$Reg(T) = TE[r(a*)] - \sum_{t=1}^{T} r(a_t)$$

Erwartungswert beste Aktion

Tatsächlich durchgeführte Aktion

- Viele relativ einfache Strategien
- Oft theoretische Garantien für Regret
  - Viele optimale Algorithmen (modulo konstantem Faktor)
  - Empirische Performance unterschiedlich
- Explorationsstrategien für komplexere MDP werden durch diese Strategien inspiriert

Optimistische Exploration: Upper Confidence Bound (UCB)

- Durchschnittlichen Reward $\hat{\mu}_a$ für jede Aktion a tracken
- Exploitation: wähle $a = \operatorname{argmax} \mu_a$
- Optimistische Schätzung: $a = \operatorname{argmax}(\mu_a + C\underbrace{\sigma_a}_{\text{irgendeine Varianzschätzung}})$

Intuition: Versuche jeden Arm bis es *sicher* ist, dass er nicht gut ist

- Beispiel (Auer et al. (2002). Finite-tie analysis of the multiarmed bandit problem.):

$$a = \operatorname{argmax}\left(\mu_a + \sqrt{\frac{2 \ln T}{N(a)}}\right) \rightarrow Reg(T) = O(\log T),$$

Beweisbar optimal (im Bandit-Setup)

Bei Unsicherheit → Optimismus

Thompson sampling

- Annahme: $r(a_i) \sim p_{\theta_i}(r_i)$,
- Das definiert einen POMDP mit $s = [\theta_1, \dots, \theta_n]$
- "belief state" ist $\hat{p}(\theta_1, \dots, \theta_n)$ (das ist ein *Modell* des Bandits)

Idee:

- Sample $\theta_1, \dots, \theta_n \sim \hat{p}(\theta_1, \dots, \theta_n)$
- Nehme an, $\theta_1, \dots, \theta_n$ sei korrekt
- Nehme optimale Aktion
- Update Modell aus Beobachtung

- Bekannt als Thompson sampling, posterior sampling oder probability matching
- Schwerer theoretisch zu analysieren
- Funktioniert gut empirisch

Siehe Chapelle & Li, "An Empirical Evaluation of Thompson Sampling"

Information gain

- Angenommen, wir möchten eine latente Variable z bestimmen
  - z könnte bspw. die optimale Aktion oder der Wert einer Aktion sein
- Welche Aktion sollten wir nehmen?
- Def.: $\mathcal{H}\big(\hat{p}(z)\big)$ ist die Entropie unserer aktuellen Schätzung von z
- Def.: $\mathcal{H}(\hat{p}(z)|y)$ ist die Entropie unserer Schätzung von z nach Beobachtung y (y könnte z.B. $r(a)$ sein)
- Je geringer die Entropie, desto präziser kennen wir z

- $IG(z, y) = \mathbb{E}_y\big[\mathcal{H}\big(\hat{p}(z)\big) - \mathcal{H}(\hat{p}(z)|y)\big]$
- Typischerweise hängt das von Aktion a ab, also $\mathrm{IG}(z, y|a)$

Information gain Beispiel

- $\mathrm{IG}(z, y|a) = \mathbb{E}_y\left[\mathcal{H}\left(\hat{p}(z)\right) - \mathcal{H}(\hat{p}(z)|y)|a\right]$
- Wie viel lernen wir über z aus Aktion a, gegeben unsere aktuellen Beliefs

- Beispiel eines Bandit-Algorithmus: Russo & Van Roy "Learning to Optimize via Information-Directed Sampling"

$y = r_a,\ z = \theta_a$ (θ_a sind die Parameter des Modells $p(r_a)$)

$g(a) = IG(\theta_a, r_a|a)$ - information gain von a

$\Delta(a) = \mathbb{E}[r(a^*) - r(a)]$ - erwartete Suboptimalität von a

Wähle a gemäß: $\mathrm{argmin}_a \frac{(\Delta(a)^2)}{g(a)}$

Wähle keine sicher suboptimalen Aktionen

Wähle keine Aktionen, aus denen du nichts lernst

Leitideen der Exploration

UCB

$$a = \text{argmax}\left(\mu_a + \sqrt{\frac{2 \ln T}{N(a)}}\right)$$

Thompson sampling

$$\theta_1, \dots, \theta_n \sim \hat{p}(\theta_1, \dots, \theta_n)$$

$$a = \text{argmax}_{\text{a}} \mathbb{E}_{\theta_a}[r(a)]$$

Information gain

$$\text{IG}(z, y|a)$$

- Die meisten Explorationsstrategien benötigen eine Art Unsicherheitsabschätzung (auch wenn sie sehr grob ist)
- Üblicherweise wird angenommen, dass neue Information einen Wert hat:
  - Unbekanntes = potenziell gut (Optimismus)
  - Stichprobe = Wahrheit
  - Informationsgewinn = gut

Warum sollten wir uns dafür interessieren?

- Bandits sind leichter zu analysieren und zu verstehen
- Sie liefern grundlegende Einsichten für Explorationsmethoden
- Diese Methoden lassen sich anschließend auf komplexere MDPs anwenden

Hier nicht behandelt:

- Kontextuelle Bandits (Bandits mit Zustand, im Grunde 1-Schritt-MDPs)
- Optimale Exploration in kleinen MDPs
- Bayesianisches modellbasiertes Reinforcement Learning (ähnlich wie Informationsgewinn)
- Probably Approximately Correct (PAC) Exploration

Exploration in Deep RL

Verschiedene Explorations-Methoden

- Optimistische Exploration
  - Neuer Zustand = guter Zustand
  - Erfordert Zustandscounter oder Neuigkeit von Zuständen messen
  - Typischerweise realisiert über Explorationsbonus
- Thompson sampling
  - Lerne Verteilung über Q-Functions oder Policies
  - Sample und agiere gemäß Sample
- Information gain
  - Erwäge Information gain durch Besuch neuer Zustände

Optimistische Exploration in Deep RL

Optimistische Exploration in Deep RL

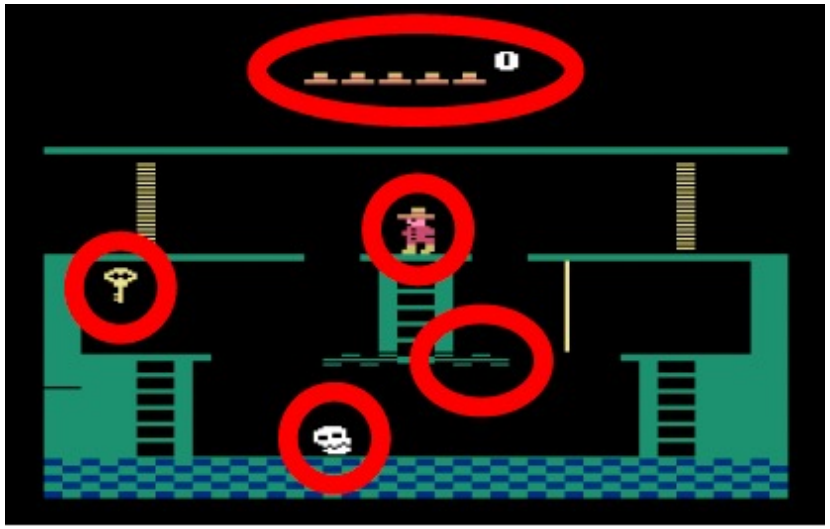
UCB: $$a = \operatorname{argmax}\left(\mu_a + \underline{\sqrt{\frac{2\ln T}{N(a)}}}\right)$$

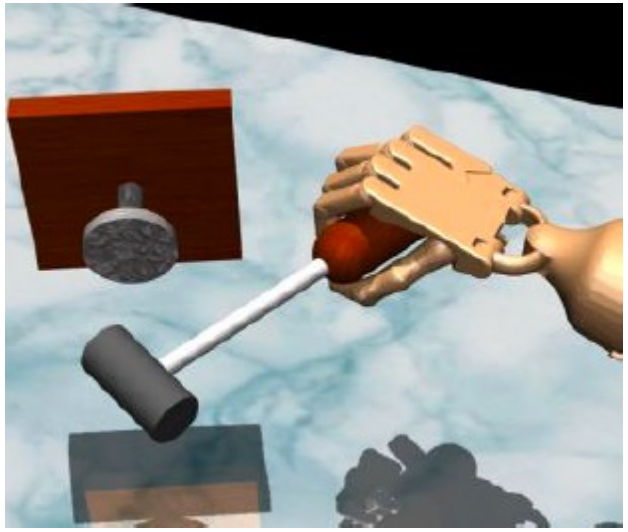
„Explorationsbonus“
Viele Funktionen möglich, solange sie mit $N(a)$ abnehmen

- Können wir das in deep RL verwenden?
- Count-basierte Exploration: verwende $N(s,a)$ oder $N(s)$ für Explorationsbonus:
- $r^+(s,a) = r(s,a) + B(N(s)),\quad B\big(N(s)\big)$: Bonus, der mit $N(s)$ abnimmt
- Nutze $r^+(s,a)$ statt $r(s,a)$ mit beliebigem modellfreien Algorithmus
- → Einfache Ergänzung für jeden RL Algorithus
- →Nachteil: Tuning von Bonusgewichten notwendig

Wie zählen wir Zustände in Deep RL?

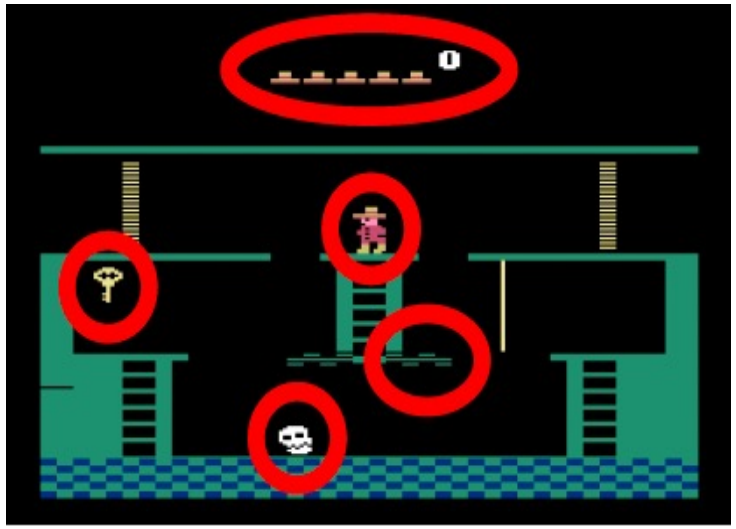
- Nutze $r^{+}(s,a) = r(s,a) + B(N(s))$
- Aber wie zählen wir Zustände in riesigen Zustandsräumen?

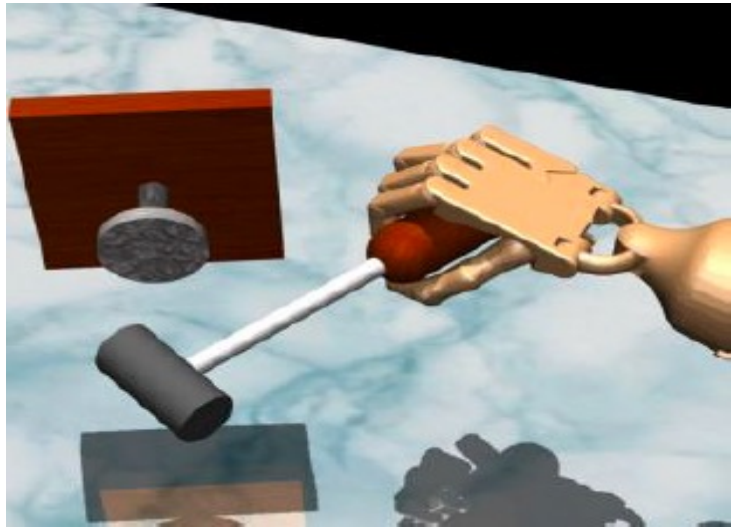




- Viele Zustände sehen wir nur ein einziges Mal!
- Aber einige Zustände ähneln einander mehr als andere…

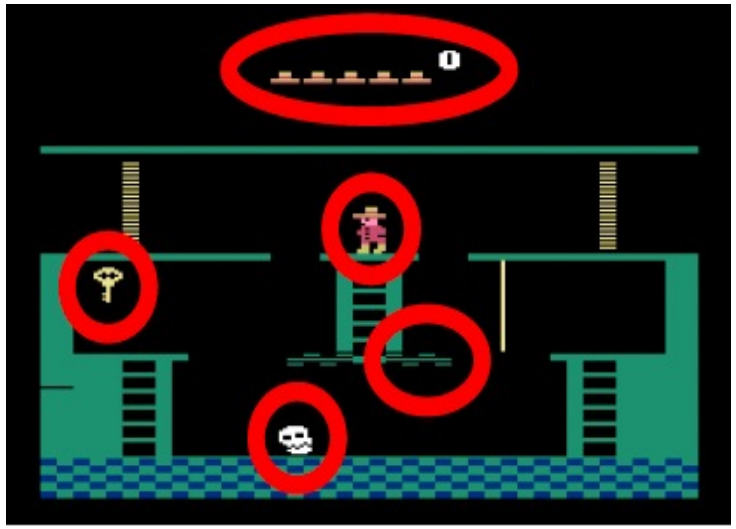
Können Modelle fitten

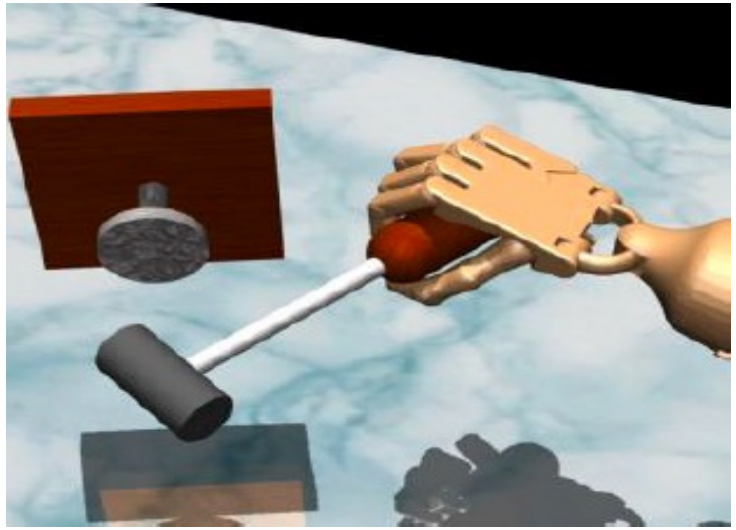




- Idee: Fitte Modell für Zustandsdichte $p_\theta(s)$ oder $p_\theta(s,a)$ als Näherung für $N(s)$ bzw. $N(s,a)$
- $p_\theta(s)$ hoch → s ist ähnlich zu einem zuvor schon gesehenen Zustand
- Wir können $p_\theta(s)$ als "Pseudo-Count" nutzen
- Für kleine MDPs: $P(s) = \frac{N(s)}{n}$
- Wenn wir s nochmal sehen: $P'(s) = \frac{N(s)+1}{n+1}$
- Idee: Wenn $p_\theta(s)$ und $p_{\theta'}(s)$ diesen Gleichungen gehorchen, können wir daraus $N(s)$ und n bestimmen

Exploration mit Pseudo-Counts





- Fitte Modell $p_\theta(s)$ an alle bisher gesehenen Zustände $\mathcal{D}$
- Mache einen Schritt i und beobachte s_i
- Fitte neues Modell $p_{\theta'}(s)$ an $\mathcal{D} \cup s_i$
- Nutze $p_\theta(s)$ und $p_{\theta'}(s)$ zur Schätzung von $\widehat{N}(s)$
- $r_i^+ = r_i + B(\widehat{N}(s)),$ $\widehat{N}(s)$: Pseudo-Count
- Wie erhalten wir $\widehat{N}(s)$? Nutze:

$$p_\theta(s_i) = \frac{\widehat{N}(s_i)}{\hat{n}} \qquad p_{\theta'}(s_i) = \frac{\widehat{N}(s_i)+1}{\hat{n}+1}$$

- Zwei Gleichungen mit zwei Unbekannten

$$\rightarrow \widehat{N}(s_i) = \hat{n}\, p_\theta(s_i) \qquad \hat{n} = \frac{1-p_{\theta'}(s_i)}{p_{\theta'}(s_i)-p_\theta(s_i)} p_\theta(s_i)$$

Bellemare et al. “Unifying Count-Based Exploration…”

Welchen Bonus nutzen?

Viele verschiedene Funktionen in der Literatur, inspiriert durch optimale Methoden für Bandits oder kleine MDPs

UCB:

$$B\big(N(s)\big) = \sqrt{\frac{2\ln n}{N(s)}}$$

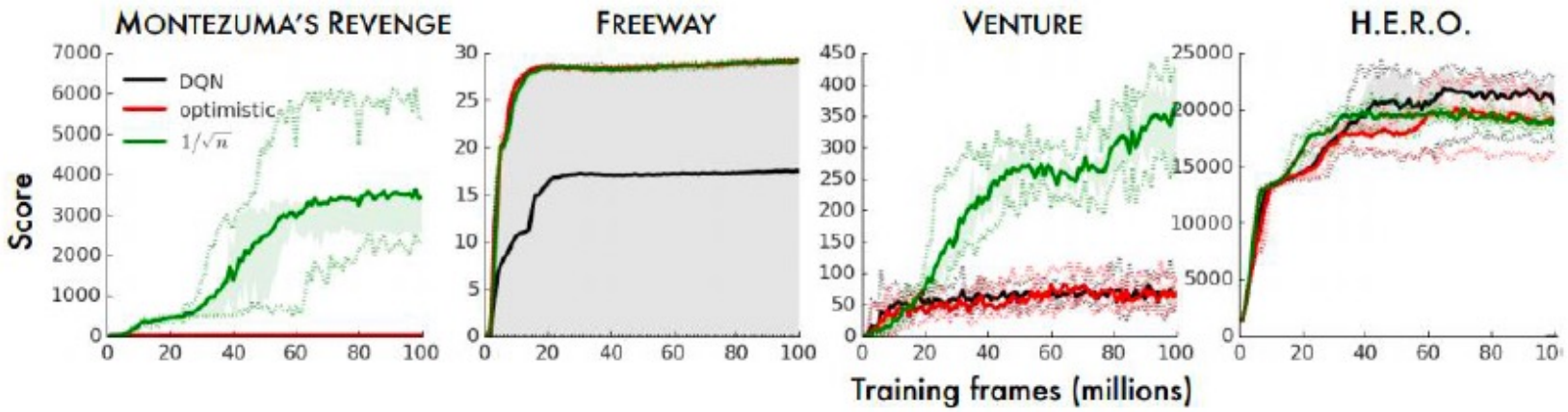
MBIE-EB (Strehl & Littmann, 2008):

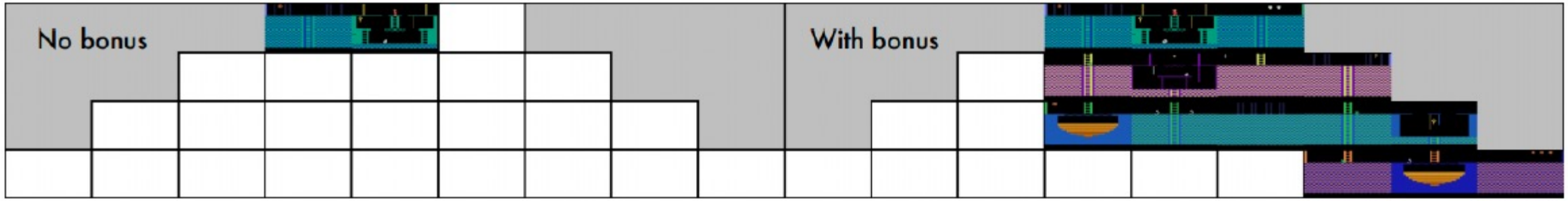
$$B\big(N(s)\big) = \sqrt{\frac{1}{N(s)}}$$

BEB (Kolter & Ng, 2009):

$$B\big(N(s)\big) = \frac{1}{N(s)}$$

Funktioniert es?

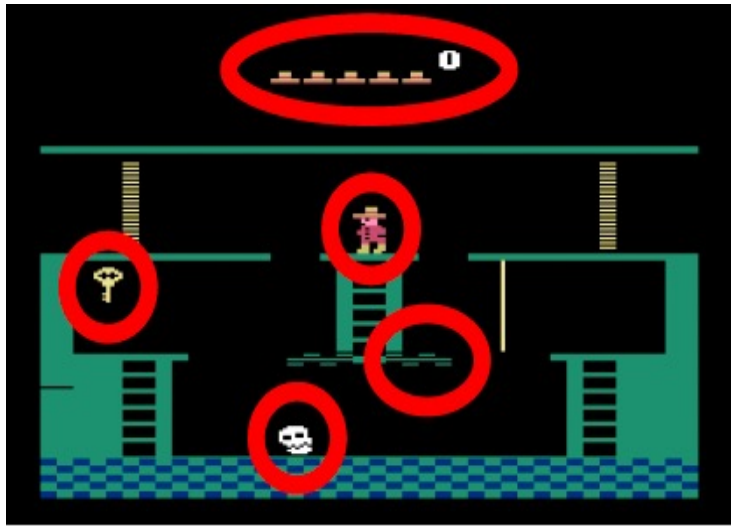


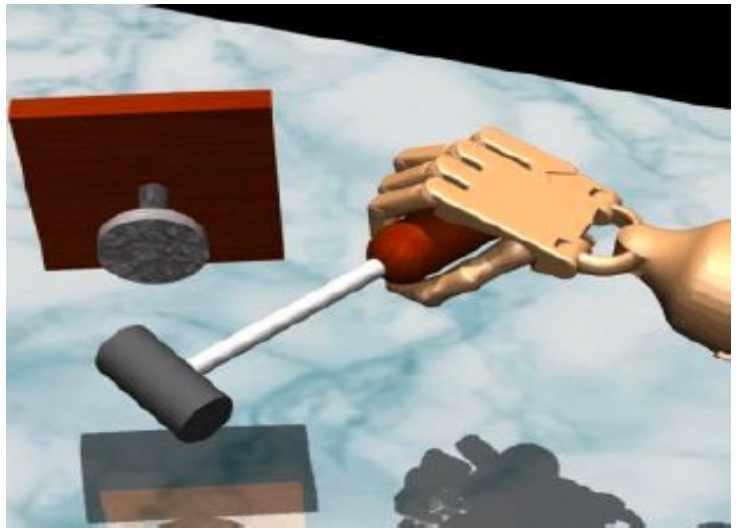


Bellemare et al. "Unifying Count-Based Exploration…"

Weitere Neuigkeit-suchende Explorationsmethoden

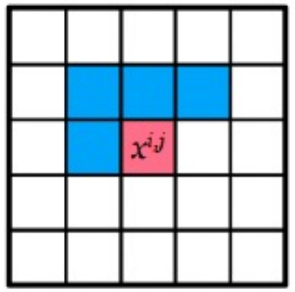
Was für ein Modell nutzen?





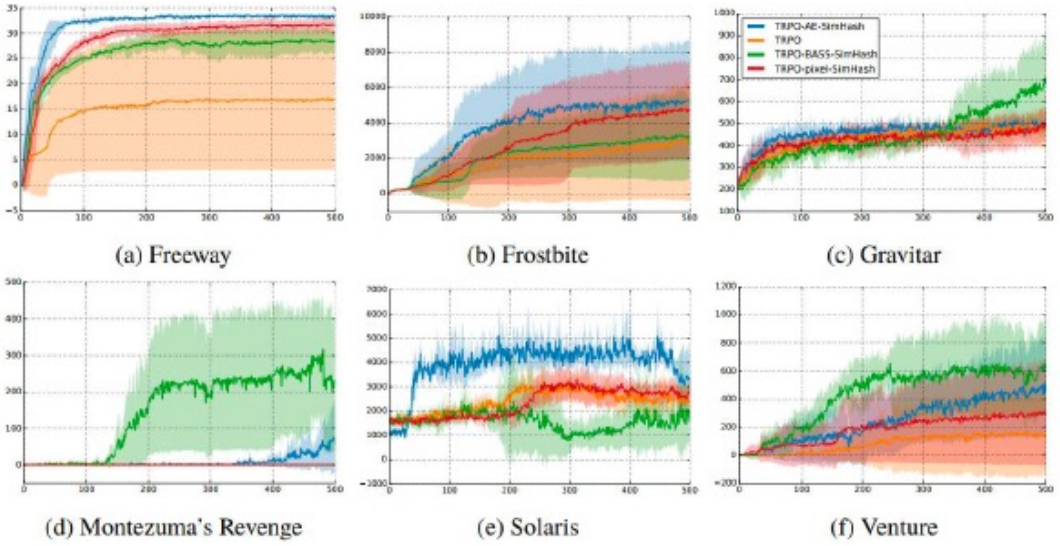
$$p_\theta(s)$$

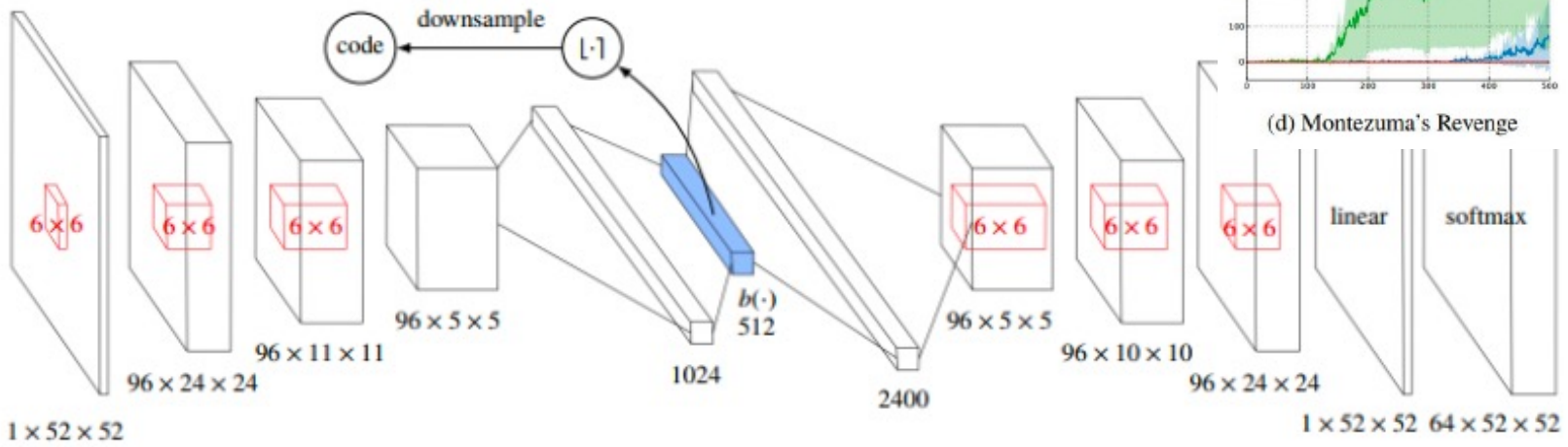
- Brauchen Zustandsdichten, aber müssen nicht unbedingt gute Samples produzieren können
- Andere Anforderungen als bei vielen generativen Modellen (GANs, Diffusion)
- Bellemare et al.: ”CTS”-Modell: Konditionierung jedes Pixels auf dessen Nachbarschaft links/oben
- Andere Modelle: Stochastische neuronale Netzwerke, Compression Length, EX2



Zählen mit Hashes

- Idee: Komprimiere s in einen k-bit Code mit $\phi(s)$ und Zähle $N\big(\phi(s)\big)$
- Kürzere Codes = mehr Hashkollisionen
- Erhalten ähnliche Zustände den selben Hash? Kommt drauf an…
- Beeinflusse dies durch Lernen einer Kompression mit Autoencodern





Tang et al. "#Exploration: A Study of Count-Based Exploration"

Implizite Dichtemodellierung mit Exemplar-Modellen

- $p_\theta(s)$: Brauchen Zustandsdichten, aber müssen nicht unbedingt gute Samples produzieren können
- Können wir den neuen Zustand explizit mit den alten Zuständen vergleichen?
- Intuition: der Zustand ist **neu**, wenn es **leicht** ist ihn mittels Classifier von allen zuvor gesehenen Zuständen zu unterscheiden
- Für jeden Zustand, fitte Classifier um diesen Zustand gegen alle vorherigen Zustände $\mathcal{D}$ zu klassifizieren; nutze dies, um eine Dichte zu erhalten:

$$p_\theta(s) = \frac{1 - D_s(s)}{D_s(s)}$$

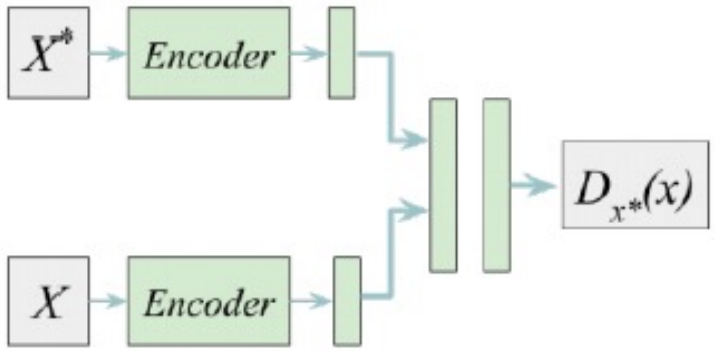
positives: $\{s\}$
negatives: $\mathcal{D}$

- $D_s(s)$: Wahrscheinlichkeit, die der Classifier für "s ist positiv" angibt
- Task ist nicht trivial, da s auch in $\mathcal{D}$ enthalten sein kann

Fu et al. "EX2: Exploration with Exemplar Models…"

Implizite Dichtemodellierung mit Exemplar-Modellen

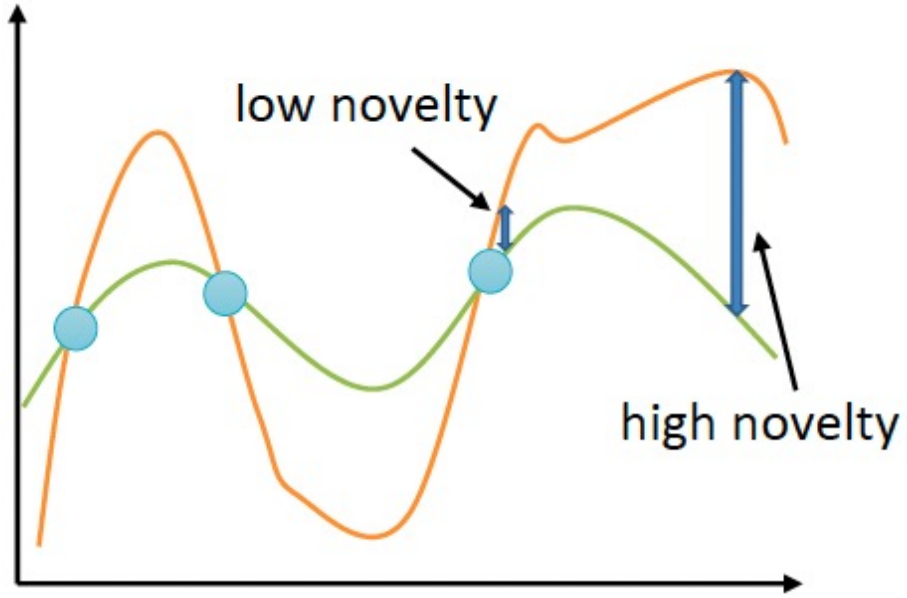
- Task ist nicht trivial, da s auch in $\mathcal{D}$ enthalten sein kann
- Wenn $s \in \mathcal{D}$, ist das optimale $D_s(s) \neq 1$, in der Tat:
- $D_s^*(s) = \frac{1}{1+p(s)} \rightarrow p_\theta(s) = \frac{1-D_s(s)}{D_s(s)}$
- In der Realität ist jeder Zustand einzigartig (großer Zustandsraum) → Regularisierung gegen Overfitting
- Ein Classifier pro Zustand ist aufwendig…
- → Amortisiertes Modell: Einzelnes Netzwerk mit Exemplar als weiterer Input:



Heuristische Count-Schätzung via Fehler

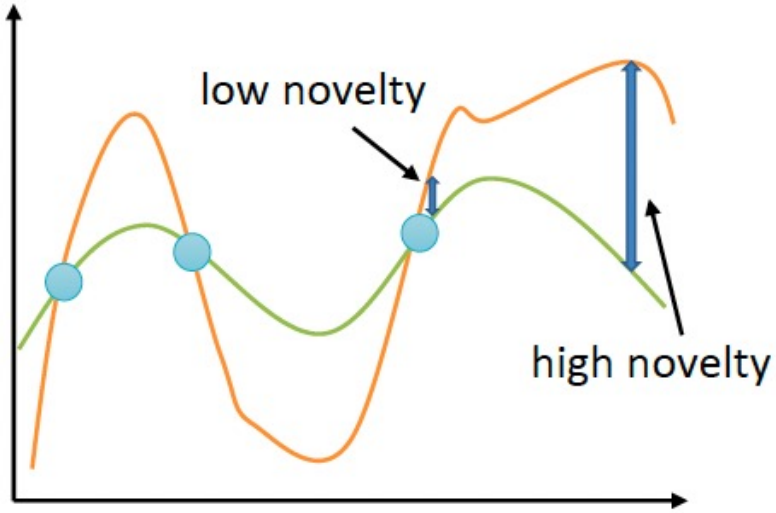
- $p_\theta(s)$: Brauchen Zustandsdichten, aber müssen nicht unbedingt gute Samples produzieren können
  - … und müssen auch nicht unbedingt gute Dichten produzieren
  - … müssen lediglich erkennen, ob ein Zustand **neu** ist oder nicht!

- Angenommen, wir haben eine Zielfunktion $f^*(s,a)$
- Fitte $\hat{f}_\theta(s,a)$ mit Daten aus unserem Buffer $\mathcal{D} = \{(s_i, a_i)\}$
- Nutze $\mathcal{E}(s,a) = \left|\hat{f}_\theta(s,a) - f^*(s,a)\right|^2$ als Bonus



Heuristische Count-Schätzung via Fehler

- Angenommen, wir haben eine Zielfunktion $f^*(s,a)$
- Fitte $\hat{f}_\theta(s,a)$ mit Daten aus unserem Buffer $\mathcal{D} = \{(s_i, a_i)\}$
- Nutze $\mathcal{E}(s,a) = \left|\hat{f}_\theta(s,a) - f^*(s,a)\right|^2$ als Bonus



Welche Zielfunktion $f^*(s,a)$ wählen?

- Oft: $f^*(s,a) = s'$, also next state prediction
- Noch einfacher: $f^*(s,a) = f_\phi(s,a)$, mit *zufälligem* Parametervektor ϕ (→ beliebiges $f^*(s,a)$)

Bruda et al. (2018), Exploration by random network distillation.

Thompson Sampling in Deep RL

Thompson Sampling in Deep RL

- Thompson Sampling:
- $\theta_1, \dots, \theta_n \sim \hat{p}(\theta_1, \dots, \theta_n)$ Was wird gesampelt?
- $a = \mathrm{argmax}_{\mathrm{a}}\, \mathbb{E}_{\theta_a}[r(a)]$ Wie wird die Verteilung repräsentiert?

- Bandit Setting: $\hat{p}(\theta_1, \dots, \theta_n)$ ist die Verteilung über *Rewards*
- MDP-Analogon ist die Q-Funktion

1. Sample Q-Funktion Q aus $p(Q)$
2. Handle gemäß Q für eine Episode
3. Update $p(Q)$

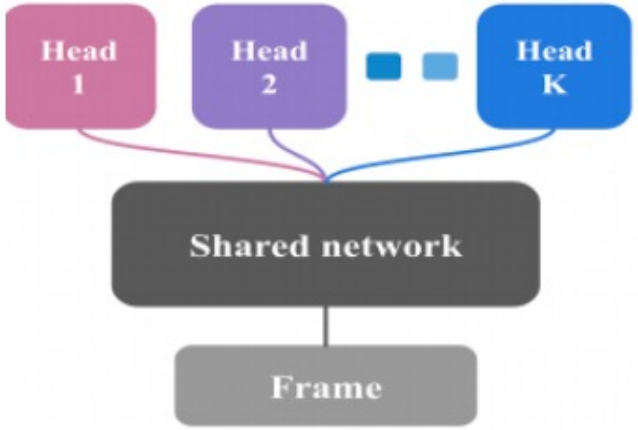
Wie können wir eine Verteilung über Funktionen ($p(Q)$) repräsentieren?

Osband et al. “Deep Exploration via Bootstrapped DQN”.

Bootstrapping $p(Q)$

- Gegeben Datensatz $\mathcal{D}$, resample N mal mit Zurücklegen →$\mathcal{D}_1, \dots, \mathcal{D}_N$
- Trainiere je ein Modell f_{θ_i} auf $\mathcal{D}_i$
- Sampeln aus $p(\theta)$ → sample $i \in [1, \dots, N]$ und nutze f_{θ_i}

- Training großer neuronaler Netzwerke ist teuer – können wir es vermeiden?

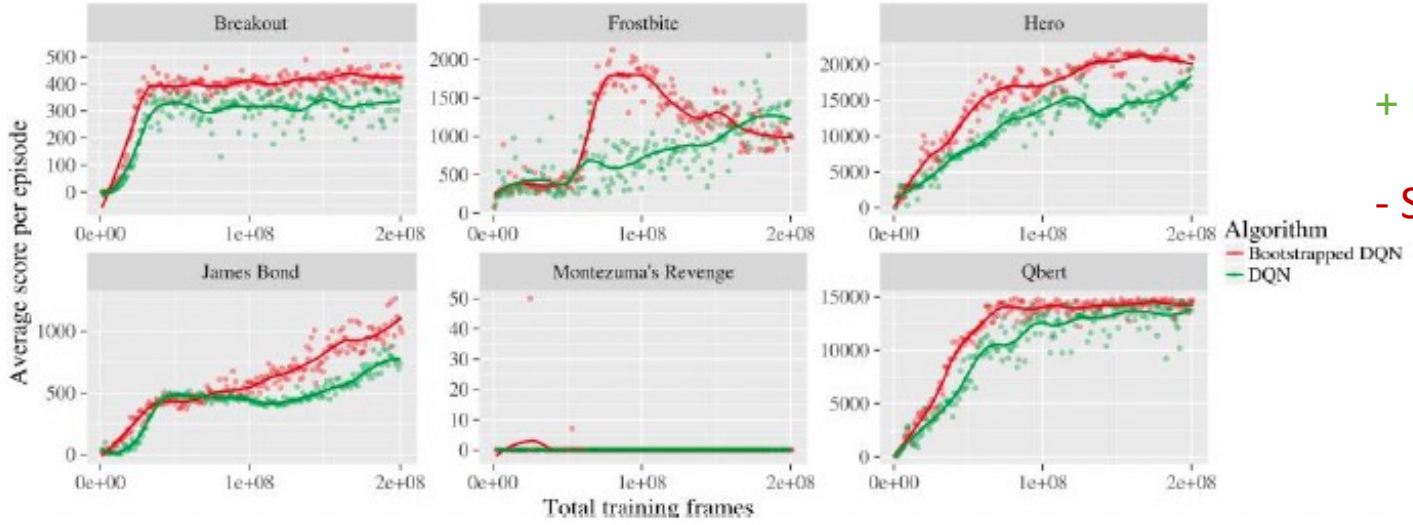


(alle auf einem Datensatz trainiert)

Osband et al. "Deep Exploration via Bootstrapped DQN".

Warum funktioniert das?

- Exploration mit Zufallsaktionen (z.B. ϵ-greedy): oszillieren hin und her, geht evtl. nicht zu interessanten Orten, keine kohärente Policy
- Exploration mit zufäligen Q-Funktionen: committe zu einer zufälligen, aber intern konsistenten Strategie für eine ganze Episode



+ keine Änderung der ursprünglichen Rewardfunktion

- Sehr gute Bonusse funktionieren oft besser

Osband et al. “Deep Exploration via Bootstrapped DQN”.

Information Gain in Deep RL

Schätzung des Information gain

- Information gain: $\mathrm{IG}(z, y|a)$

- Information gain *worüber*?
- Information gain über Reward $r(s, a)$? nicht nützlich, wenn Rewards selten
- Information gain über Zustandsdichte $p(s)$? - schon besser, macht mehr Sinn
- Information gain über Dynamik $p(s'|s, a)$? - guter Proxy um MDP zu lernen

- Grundsätzlich nicht genau berechenbar, unabhängig davon, was geschätzt wird!

Schätzung des Information gain

- Grundsätzlich nicht genau berechenbar, unabhängig davon, was geschätzt wird!

Einige Näherungen:

Prediction gain: $\log p_{\theta'}(s) - \log p_{\theta}(s)$ (Schmidhuber '91, Bellemare '16)

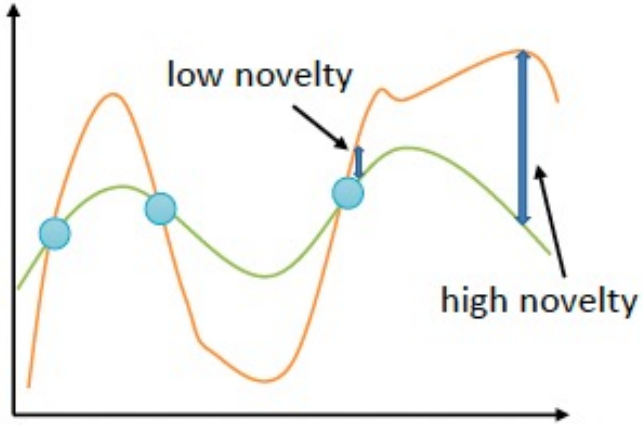
- Intuition: Wenn Dichte sich stark ändert, war der Zustand neu

Variational inference (Houthooft et al. "VIME")

- IG kann geschrieben werden als $D_{KL}(p\big(p(z|y) \parallel p(z)\big)$
- Lernen Transitions $p_\theta(s_{t+1}|s_t, a_t)$: $z = \theta, y = (s_t, a_t, s_{t+1})$
- Intuition: Transition ist informativ, wenn es unseren Belief über θ ändert
- Schätzen $p(\theta) \approx q_\phi(\theta)$ und nutzen $D_{KL}\left(q_{\phi'}(\theta) \parallel q_\phi(\theta)\right)$ als Explorations-Bonus

Exploration mit Modellfehlern

- $D_{KL}\left(q_{\phi'}(\theta)\middle|q_{\phi}(\theta)\right)$ kann auch als Änderung der Netzwerkparameter ϕ aufgefasst werden; unabhängig von IG gibt es viele Wege diese zu messen



Stadie et al 2015

- Bilder mit auto-encoder codieren
- Vorhersagemodell auf latentem Raum des Autoencoders
- Modellfehler als Explorationsbonus nutzen

Schmidhuber et al (“Formal Theory of Creativity, Fun, and Intrinsic Motivation”)

- Explorationsbonus für Modellfehler, Modellgradient, und viele andere Varianten

In Kürze: Explorationsklassen in Deep RL

Optimistische Exploration

- Exploration mit Pseudecounts basierend auf Zustandsdichten
- Verschiedene Methoden zur Schätzung von Zustandsdichten

Thompson sampling

- Verteilung über Modelle mittels Bootstrapping
- Verteilung über Q-Funktionen

Information gain Algorithmen

- IG nicht berechenbar, aber diverse Methoden zum Schätzen

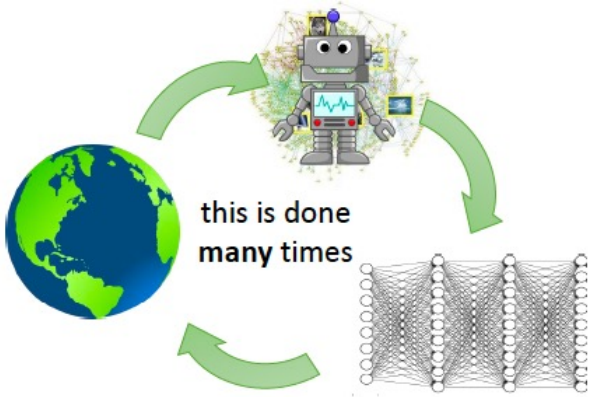
Offline Reinforcement Learning

Literatur

- Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems, https://arxiv.org/abs/2005.01643
- A Survey on Offline Reinforcement Learning: Taxonomy, Review, and Open Problems, https://arxiv.org/abs/2203.01387

Generalization Gap

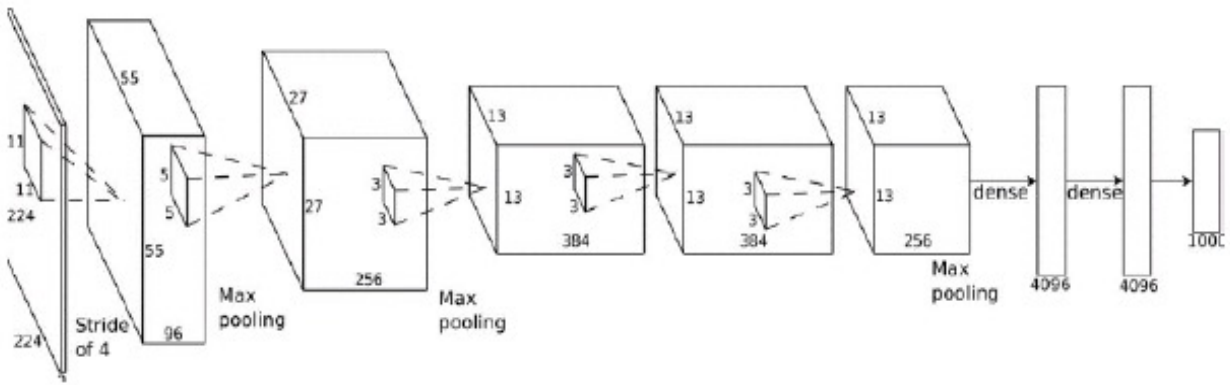
- Deep RL funktioniert auf einer großen Breite von Aufgaben
- Aber: Generalisierung nicht wie bei supervised ML
- Liegt es an der Verfügbarkeit von großen Datensätzen für RL?
- Schwierigkeit von RL: Daten werden während des Trainings gesammelt
  - On-policy: Update der Policy nur mit den neusten Daten
  - Off-policy: Update der Policy aus einem Replay-Buffer, der aber während des Trainings befüllt und ständig erneuert wird



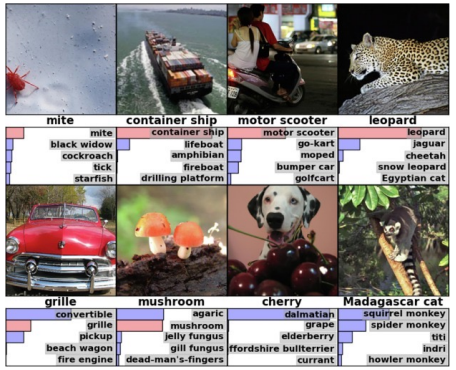
- Müssen riesige Datenmengen pro Trainingslauf sammeln
- Idealerweise: Daten einmal sammeln und wiederverwenden, wie bei Supervised Learning

Warum funktioniert Deep Learning so gut?

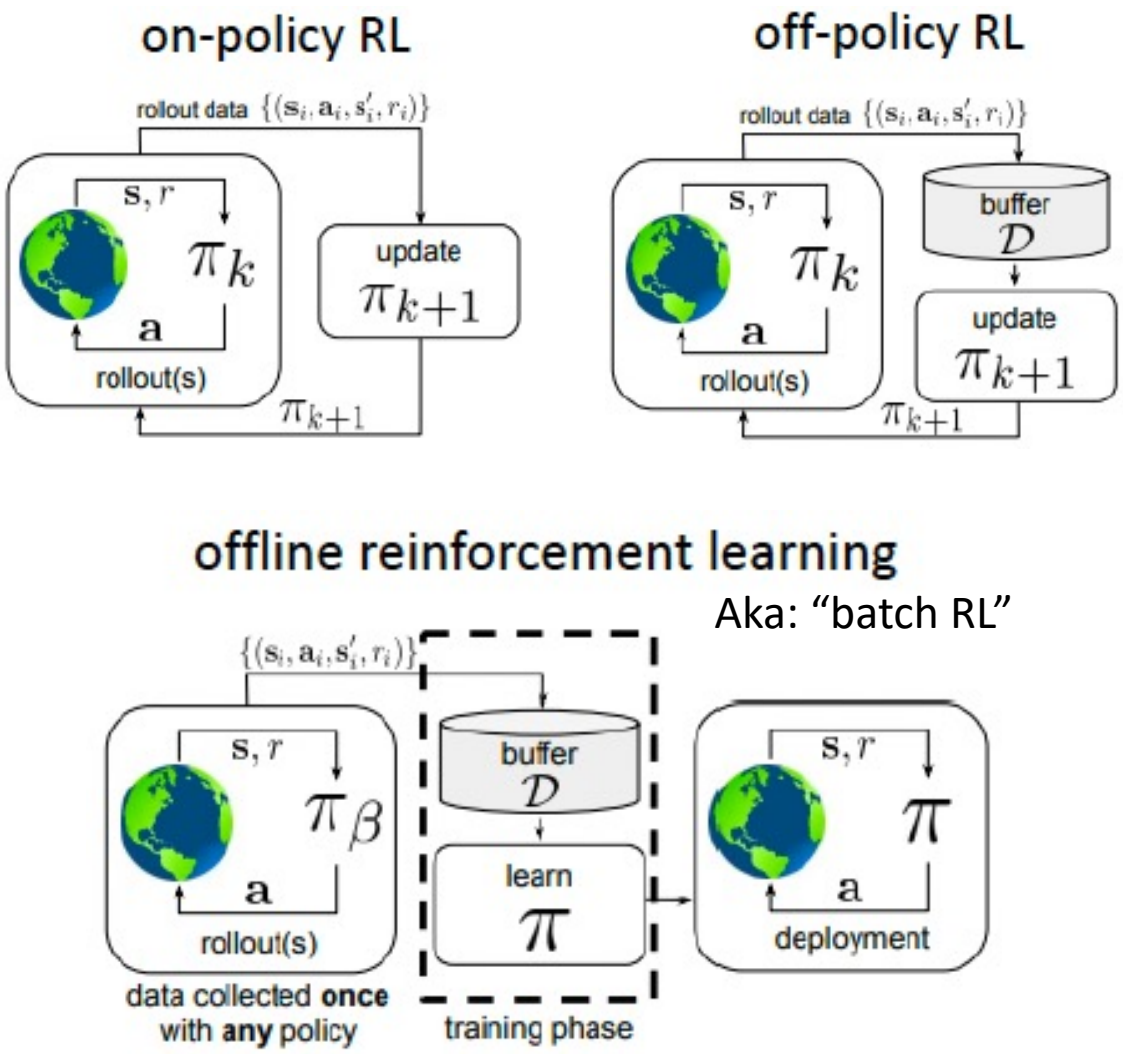
Große Datensätze

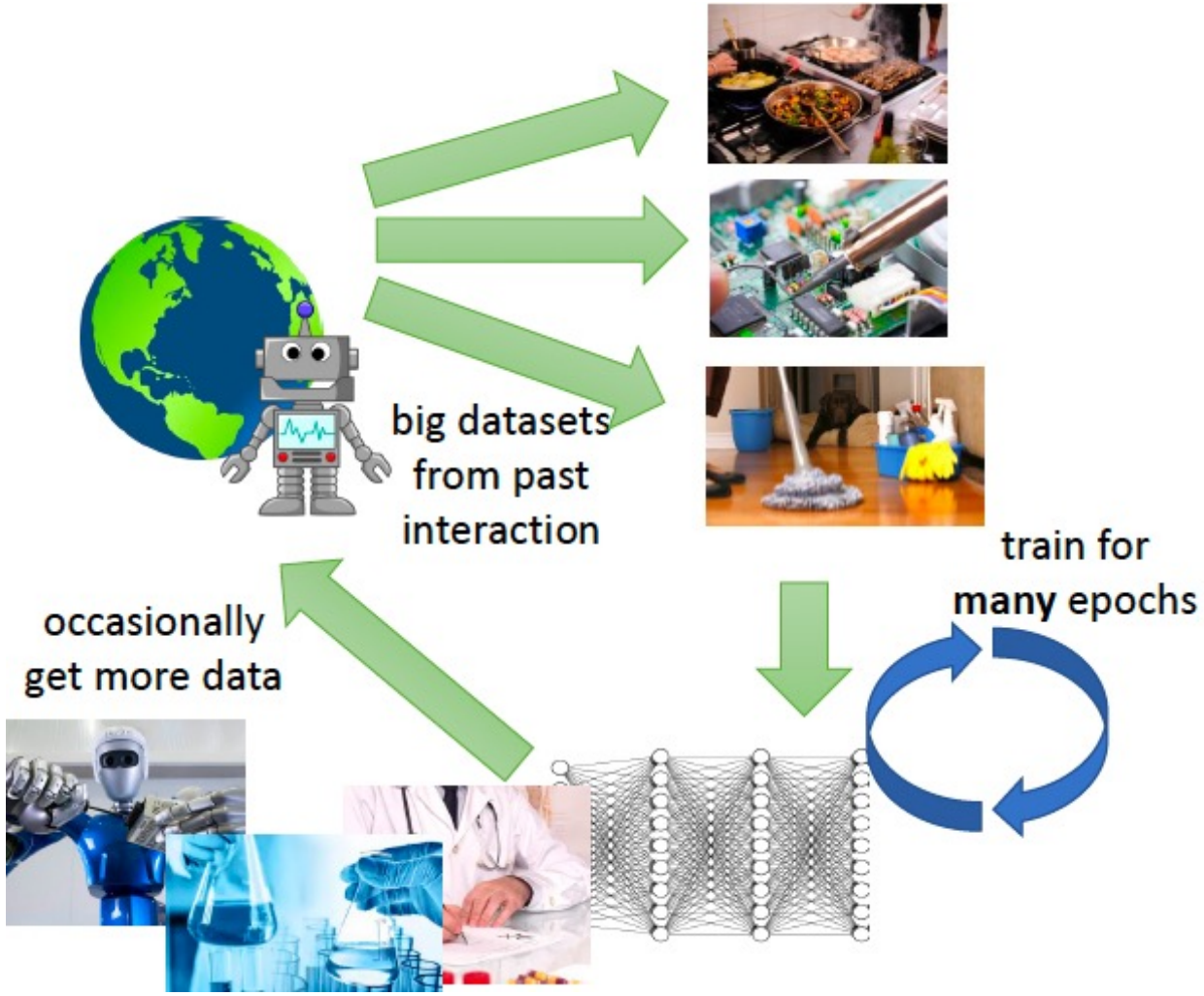


Große Modelle



Können wir datenbasierte RL-Algorithmen entwickeln?

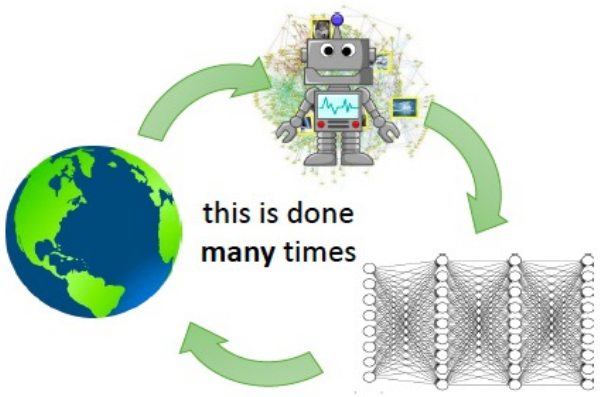




Levine, 2023

Was bringt uns Offline RL? Weitere Gründe…

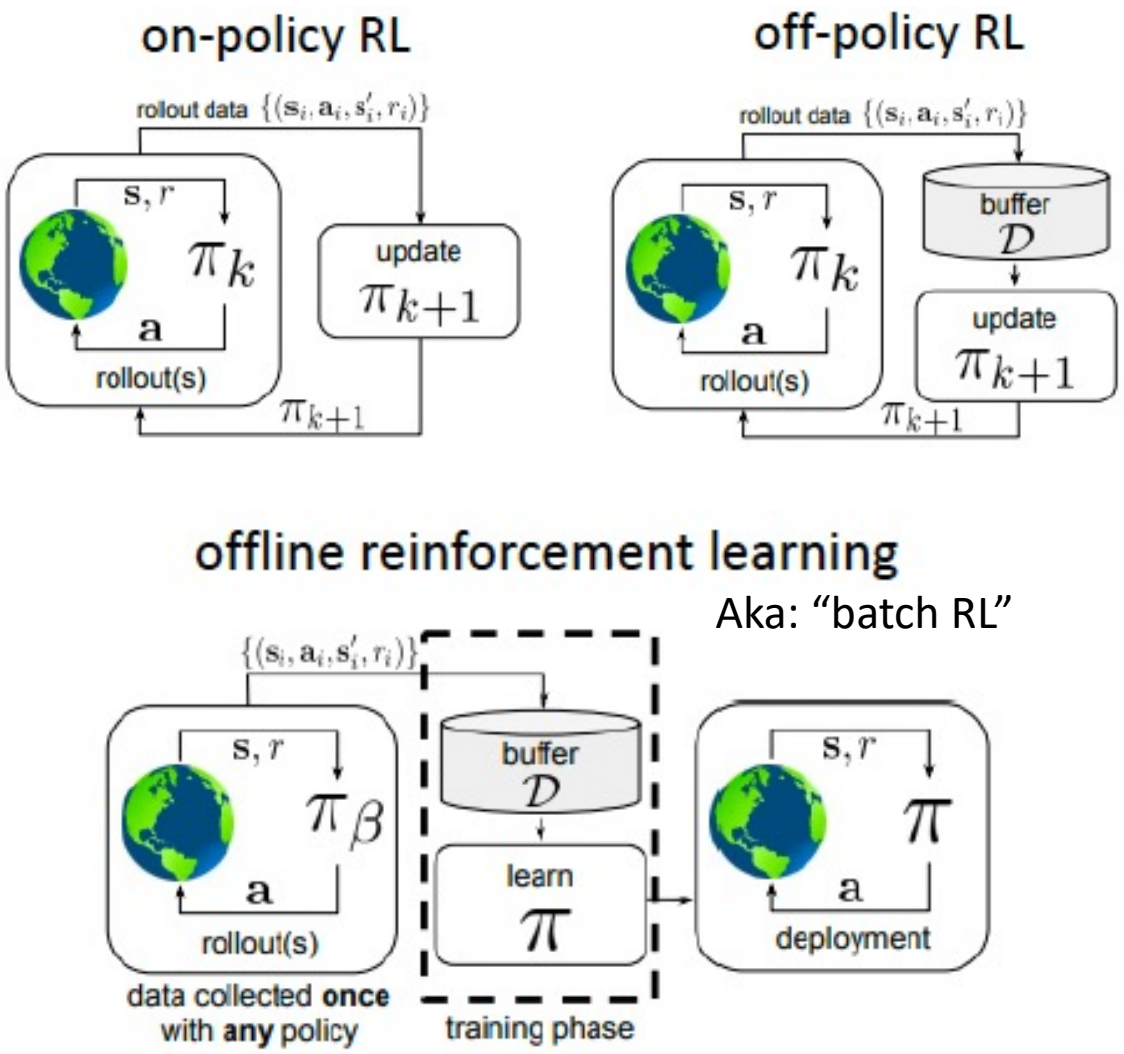
Typisches (online) RL Setup: Sammeln von Erfahrung durch Interaktion mit der Umgebung



→in vielen Bereichen nicht möglich (Sicherheit, Kosten)
z.B. Gesundheitswesen, autonomes Fahren

Offline RL ermöglicht es aus gesammelten Daten gute Policies zu extrahieren ohne riskante oder teure Interaktionen.

Offline RL: Formale Definition



$$\mathcal{D} = \{(s_i, a_i, s'_i, r_i)\}$$

$$s \sim d^{\pi_\beta}(s)$$

$$a \sim \pi_\beta(a|s)$$

unbekannt; keine Annahmen an π_β

$$s' \sim p(s'|s, a)$$

$$r = r(s, a)$$

RL Zielfunktion:

$$\max_{\pi} \sum_{t=0}^{T} \mathbb{E}_{s_t \sim d^{\pi}(s), a_t \sim \pi(a|s)}[\gamma^t r(s_t, a_t)]$$

Verschiedene Offline RL Aufgabenstellungen

Off-Policy Evaluation (OPE):

- Gegeben $\mathcal{D}$, schätze $J(\pi) = \mathbb{E}_{\pi}[\sum_{t=0}^{T} r(s_t, a_t)]$

Offline Reinforcement Learning:

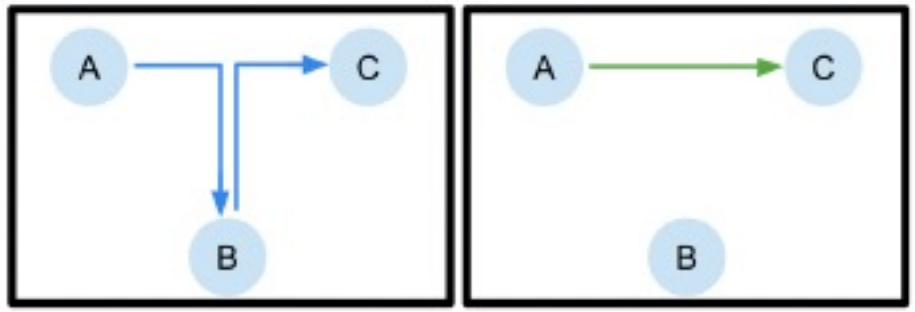
- Gegeben $\mathcal{D}$, lerne die bestmögliche Policy π_θ

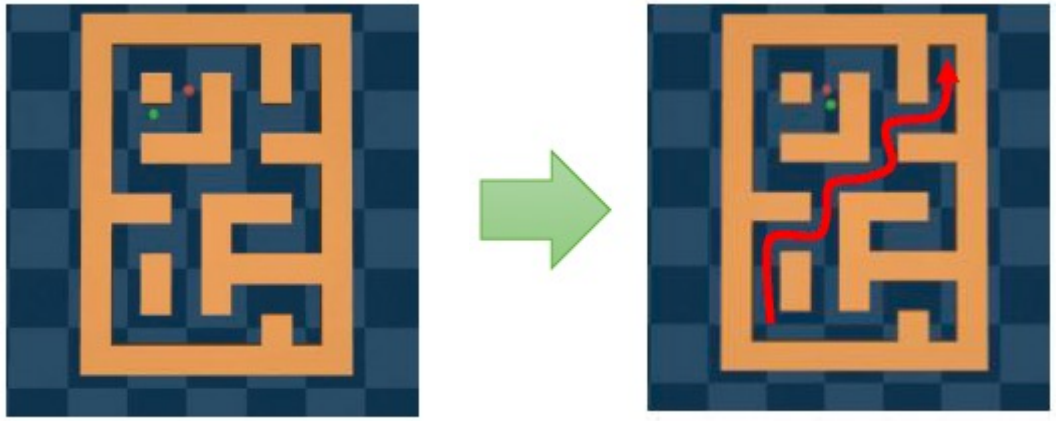
bestmöglich, gegeben $\mathcal{D}$ – nicht bestmöglich für den MDP

$$\mathcal{D} = \{(s_i, a_i, s_i', r_i)\}$$
$$s \sim d^{\pi_\beta}(s)$$
$$a \sim \pi_\beta(a|s)$$
$$s' \sim p(s'|s, a)$$
$$r = r(s, a)$$

Wie ist das überhaupt möglich?

- Finde die guten Aktionen im Datensatz voller guter und schlechter Aktionen
  - Nicht einfach Imitation Learning
- Generalisierung: Gutes Verhalten in einer Situation liefert Beispiele für gutes Verhalten in anderen Situationen.
- “Stitching”: Teile guten Verhaltens können kombiniert werden

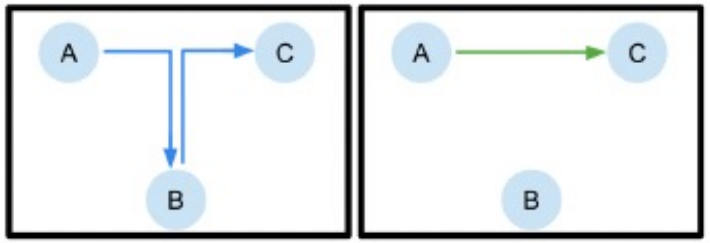




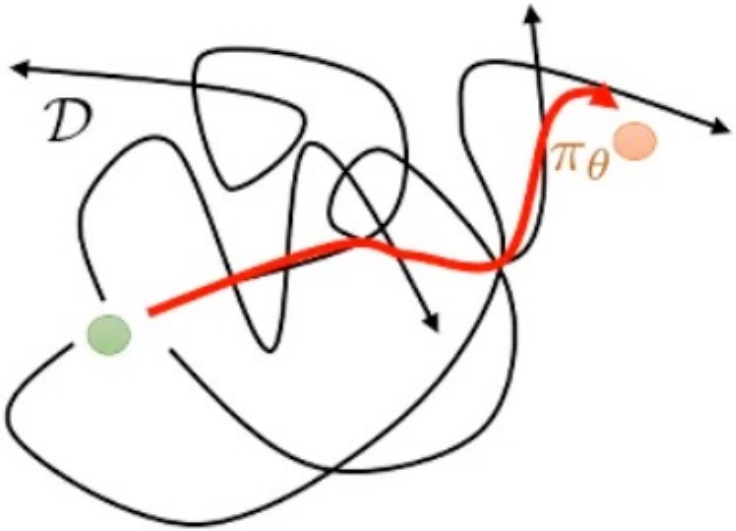
Was erwarten wir von Offline RL Methoden?

- Schlechte Intuition: es ist wie Imitation Learning
  Unter bestimmten Annahmen kann man zeigen, dass Offline RL selbst mit optimalen Daten besser ist als Imitation Learning.
  (Kumar et al. Should I run Offline RL or BC?)

- Bessere Intuition: Ordnung aus Chaos

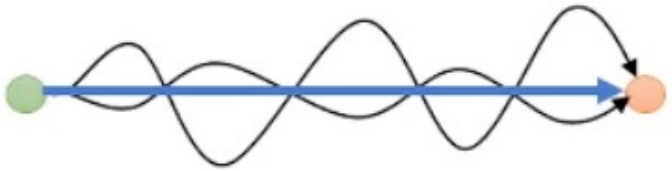


"Macro-Stitching"



Allgemeiner:

"Micro-Stitching":

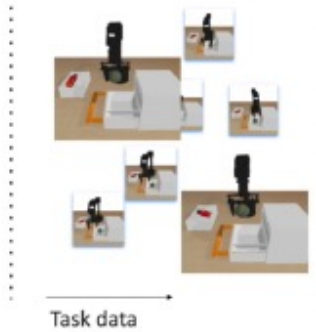


Optimale Policies aus suboptimalen Daten

Demo

RL policies typically don't generalize to initial conditions that were not seen during training

Can we use previously collected, unlabeled datasets to extend learned skills?



training task

closed drawer

blocked by drawer

blocked by object

https://www.youtube.com/watch?v=6sb31PtpI_s https://github.com/avisingh599/cog

Singh, Yu, Yang, Zhang, Kumar, Levine. **COG: Connecting New Skills to Past Experience with Offline Reinforcement Learning.** '20

Funktioniert es?

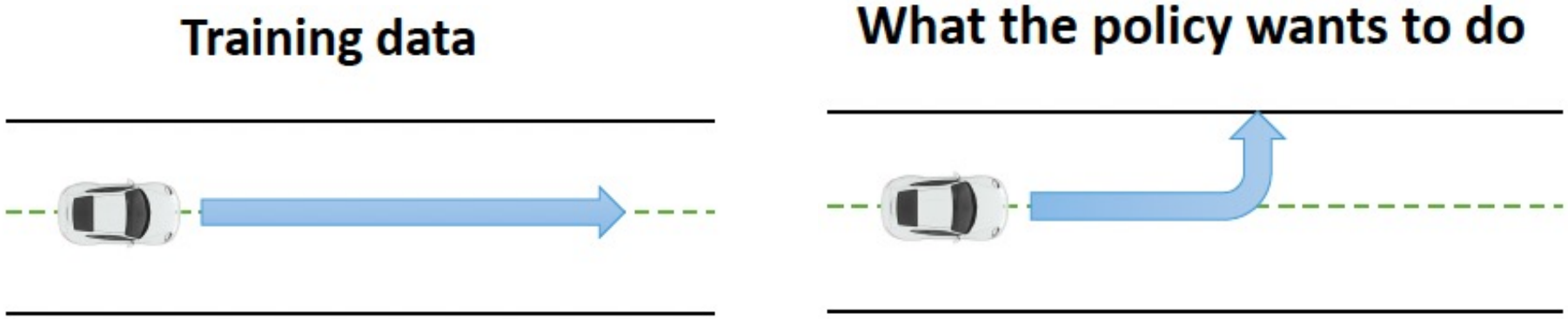
Method	Dataset	Success	Failure
Offline QT-Opt	580k offline	87%	13%
Finetuned QT-Opt	580k offline + 28k online	**96%**	**4%**

- Offline Training funktioniert, aber:
- Reines Offline-Training schnitt deutlich schlechter ab als Offline RL mit Online Finetuning

Kalashnikov et al. **QT-Opt: Scalable Deep Reinforcement Learning of Vision-Based Robotic Manipulation Skills**

Was macht Offline RL so schwierig?

Fundamentales Problem: **Counterfactual Queries.**



Ist das gut oder schlecht?

Woher soll der Agent das wissen, wenn er es nicht in den Daten gesehen hat?

Online RL Algorithmen haben das Problem nicht: Sie probieren die Aktion und schauen was passiert.

Offline RL Algorithmen müssen mit solchen ungesehenen ("out-of-distribution") Aktionen irgendwie auf sichere Art und Weise umgehen…
…und dabei dennoch generalisieren und Policies produzieren, die besser sind, als alles, was in den Daten enthalten ist.

Wichtig: out-of-distribution-actions ≠ out-of-sample-actions

Distribution shift

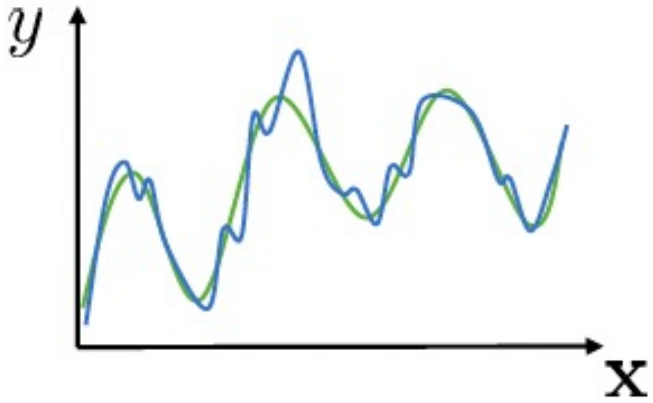
Supervised Learning:

$$\min_{\theta} \mathbb{E}_{x\sim p(x),y\sim p(y|x)}[(f_\theta(x) - y)^2]$$

Egal, neuronale Netzwerke generalisieren sehr gut!?

Problem: Adversarial Examples:

Wähle $x^* \leftarrow \mathrm{argmax}_x f_\theta(x)$



Gegeben ein x^*, ist $f(x^*)$ korrekt?

$\mathbb{E}_{x\sim p(x),y\sim p(y|x)}[(f_\theta(x) - y)^2]$ ist klein

$\mathbb{E}_{x\sim \bar{p}(x),y\sim p(y|x)}[(f_\theta(x) - y)^2]$ nicht klein für beliebige $\bar{p}(x) \neq p(x)$

Was, wenn $x^* \sim p(x)$? nicht unbedingt…

Woher kommt der Distribution Shift in RL?

$$Q(s,a) \leftarrow r(s,a) + \max_{a'} Q(s',a')$$

$$Q(s,a) \leftarrow \underbrace{r(s,a) + \mathbb{E}_{a' \sim \pi_{new}(s')}[Q(s',a')]}_{y(s,a)}$$

Offline RL:

$$\min_Q \mathbb{E}_{(s,a)\sim\pi_\beta(s,a)} \left[\left(Q(s,a) - y(s,a) \right)^2 \right]$$

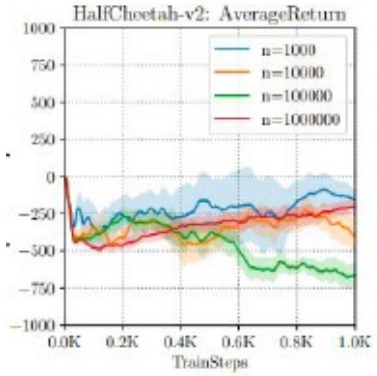
behavior policy

target

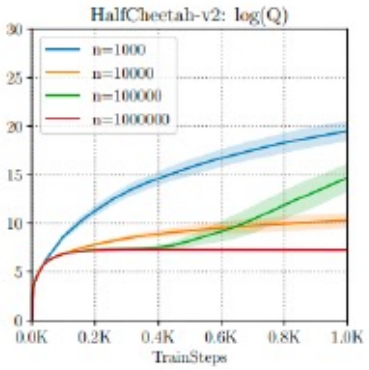
Erwarten guten Fit, wenn $\pi_\beta(a|s) = \pi_{\text{new}}(a|s)$ – aber wir wollen ja, dass π_{new} besser wird als π_β!

Sogar: $\pi_{\text{new}} = \text{argmax}_\pi \mathbb{E}_{a\sim\pi(a|s)}[Q(s,a)]$

("adversarial attack")



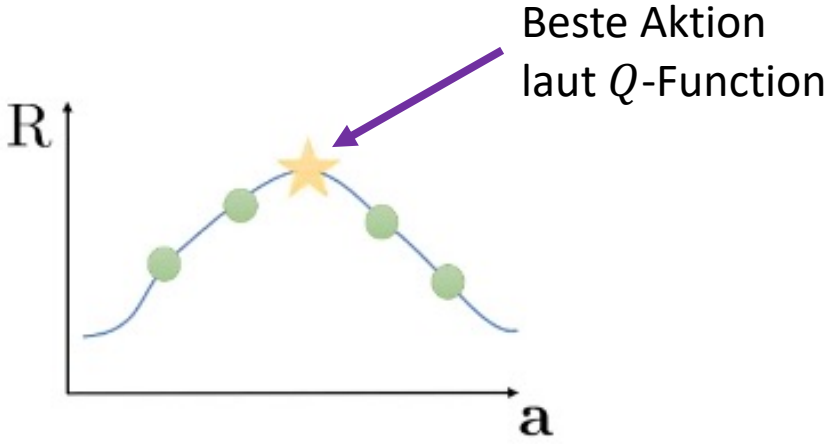
how well it does

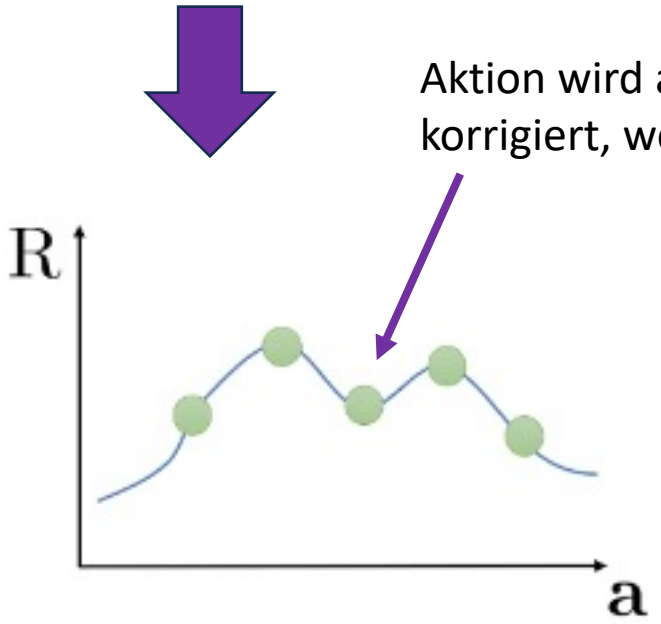


how well it *thinks* it does (Q-values)

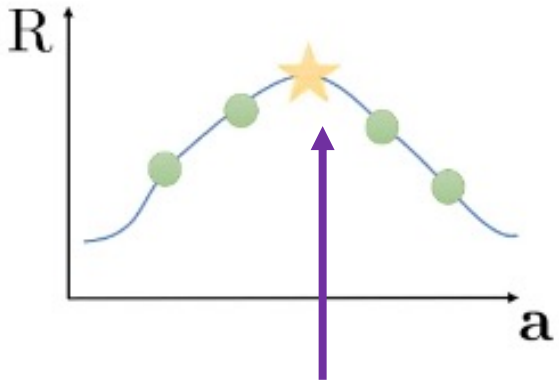
Generalisierungsfehler werden nicht korrigiert

Online RL:





Offline RL:



Keine Möglichkeit Herauszufinden, dass das keine optimale Aktion ist.

Herausforderungen mit Fehlern des Function Approximators in standard RL sind noch viel schlimmer in offline RL.

Offline RL Algorithmen

Offline RL mit Policy Gradients

$$\tau = s_1, a_1, \dots, s_T a_T, \quad r(\tau) = \sum_{t=1}^{T} r(s_t, a_t), \quad J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[r(\tau)]$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(s_t, a_t)\right) \hat{Q}(s_t, a_t)\right]$$

($\hat{Q}$: Reward to go)

$$\approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(s_{i,t} a_{i,t})\, \hat{Q}(s_{i,t} a_{i,t})$$

Müssen mit π_θ sampeln - aber wir haben nur Sampels von π_β!

Importance Sampling:

Importance Weights, exponentiell in T!
Führt zu hoher Varianz in Gradient-Schätzung.

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \left(\prod_{t'=1}^{T} \frac{\pi_\theta(s_{i,t'}, a_{i,t'})}{\pi_\beta(s_{i,t'}, a_{i,t'})}\right) \nabla_\theta \log \pi_\theta(s_{i,t}, a_{i,t})\, \hat{Q}(s_{i,t}, a_{i,t})$$

Policy Constraint Methoden

$$Q(\mathbf{s},\mathbf{a}) \leftarrow r(\mathbf{s},\mathbf{a}) + E_{\mathbf{a}'\sim\pi_{\text{new}}}[Q(\mathbf{s}',\mathbf{a}')]$$

$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \arg\max_{\pi} E_{\mathbf{a}\sim\pi(\mathbf{a}|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})] \text{ s.t. } D_{\text{KL}}(\pi\|\pi_\beta) \leq \epsilon$$

Problem 1: Kennen π_β nicht, Daten könnten kommen von

- Demonstrationen von Menschen
- einer regelbasierten Steuerung
- Viele frühere RL Läufe
- Eine Kombination von allem

Möglichkeiten für Problem 1:

- Können ein Modell für π_β fitten (Behavioral Cloning)
- Methoden, die mit Samples auskommen

Problem 2: D_{KL} Constraint ist gleichzeitig ***zu pessimistisch*** und ***nicht pessimistisch genug***

- Nicht pessimistisch genug, weil selbst für $D_{KL} < \epsilon$ einzelne Vorhersagen sehr unterschiedlich sein können
- Zu pessimistisch, weil wir eine Policy möchten, die deutlich besser als π_β ist
  - Bsp.: wenn π_β uniforme Zufallspolicy, zwingt D_{KL} Constraint die neue Policy ebenfalls nah an uniform zufällig zu sein

Explizite Policy Constraints

Welche Formen sind möglich?

- KL-Divergenz: $D_{KL}(\pi \parallel \pi_\beta)$
  - Vorteil: Leicht zu implementieren
  - Nachteil: Nicht unbedingt was wir wollen
- Support Constraint: $\pi(a|s) \geq 0$ nur, wenn $\pi_\beta(a|s) \geq \epsilon$
  - Vorteil: viel näher dran an dem, was wir wirklich wollen
  - Nachteil: deutlich aufwendiger zu implementieren

Beispielalgorithmen / verwandte Verfahren:
BRAC (Wu et al., 2019) (KL-basierter Constraint)
BEAR (Kumar et al., 2019) (Support-Constraint)
BCQ (Fujimoto et al., 2019) (Action-/Support-Constraint)
TD3+BC (Fujimoto & Gu, 2021) (BC-regularisiertes TD3, minimalistische Baseline mit oft sehr guten Ergebnissen)

Implizite Policy Constraints Methoden

$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \arg\max_{\pi} E_{\mathbf{a}\sim\pi(\mathbf{a}|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})] \text{ s.t. } D_{\text{KL}}(\pi\|\pi_\beta) \leq \epsilon$$

Lagrange-Multiplikatoren ➡

$$\pi^*(a|s) = \frac{1}{Z(s)}\pi_\beta(a|s)\exp\left(\frac{1}{\lambda}A^{\pi^*}(s,a)\right)$$

D.h., optimale Lösung ist Behavior-Policy π_β gewichtet mit exp von Advantage-Term $A(s,a)$. → Bessere Aktionen erhalten in der optimalen Policy exponentiell mehr Gewicht gegenüber als schlechtere Aktionen.

Problem: Wir kennen weder $\pi_\beta(a|s)$ noch $A^{\pi^*}(s,a)$.

Idee: Nähere die Lösung an durch iteratives Max-Likelihood-Training mit entsprechenden Gewichten:

$$\pi_{\text{new}}(a|s) = \text{argmax}_\pi \mathbb{E}_{(s,a)\sim\pi_\beta}\left[\log\pi(a|s)\overbrace{\frac{1}{Z(s)}\pi_\beta(a|s)\exp\left(\frac{1}{\lambda}A^{\pi_{\text{old}}}(s,a)\right)}^{w(s,a)}\right]$$

Trainingsdaten aus dem Datensatz

Training eines Critics um $A^{\pi_{\text{old}}}$ zu erhalten

"Gewichtetes Behavioral Cloning"

Implizite Policy Constraints Methoden

Loss von Critic (→ $Q(s,a)$) und Actor ($\pi(s,a)$):

$$\mathcal{L}_C(\phi) = E_{(\mathbf{s},\mathbf{a},\mathbf{s}')\sim D}\left[\left(Q_\phi(\mathbf{s},\mathbf{a}) - (r(\mathbf{s},\mathbf{a}) + \gamma E_{\mathbf{a}'\sim\pi_\theta(\mathbf{a}'|\mathbf{s}')}[Q_\phi(\mathbf{s}',\mathbf{a}')])\right)^2\right]$$

$$\mathcal{L}_A(\theta) = -E_{(\mathbf{s},\mathbf{a})\sim\pi_\beta}\left[\log\pi_\theta(\mathbf{a}|\mathbf{s})\frac{1}{Z(\mathbf{s})}\exp\left(\frac{1}{\lambda}A^{\pi_{\text{old}}}(\mathbf{s},\mathbf{a})\right)\right]$$

1. $\phi \leftarrow \phi - \alpha\nabla_\phi\mathcal{L}_C(\phi)$
2. $\theta \leftarrow \theta - \alpha\nabla_\theta\mathcal{L}_A(\theta)$

$\sim$

$$Q(\mathbf{s},\mathbf{a}) \leftarrow r(\mathbf{s},\mathbf{a}) + E_{\mathbf{a}'\sim\pi_{\text{new}}}[Q(\mathbf{s}',\mathbf{a}')]$$

$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \arg\max_\pi E_{\mathbf{a}\sim\pi(\mathbf{a}|\mathbf{s})}[Q(\mathbf{s},\mathbf{a})] \text{ s.t. } D_{\text{KL}}(\pi\|\pi_\beta) \leq \epsilon$$

λ: Kontrolliert die Stärke des D_{KL}-Constraints (Hyperparameter)

$Z(s)$: verschiedene Möglichkeiten, am einfachsten: “weglassen” ($Z(s) = 1$)

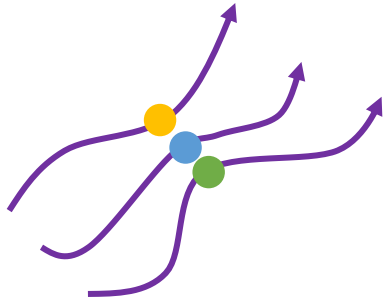
Algorithmen:
AWR (Advantage weighted regression), https://arxiv.org/abs/1910.00177
AWAC (Advantag weighted actor critic), https://arxiv.org/abs/2006.09359
CRR (Critic Regularized Regression), https://arxiv.org/abs/2006.15134

Hinweis: Herleitung nimmt explizit stochastische Policy an.

Können wir auch im Q-Update OOD-Aktionen vermeiden?

$$Q(s,a) \leftarrow r(s,a) + \underbrace{\mathbb{E}_{a' \sim \pi_{\text{new}}}[Q(s',a')]}_{V(s')}$$

(weiteres Netzwerk)



Erhalten $Q(s_i, a_i)$ für verschiedene a_i durch Generalisierung über ähnliche s_i

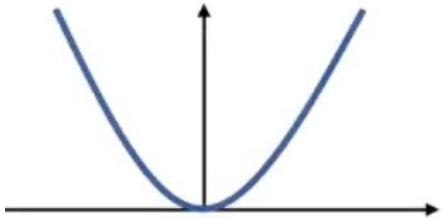
Loss zum Training von V: $L = \frac{1}{N}\sum_{i=1}^{N} l\big(V(s_i,a_i), Q(s_i,a_i)\big)$

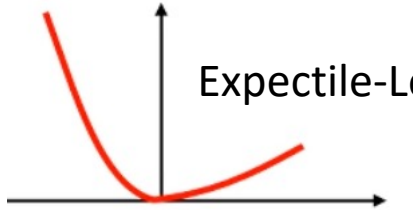
Summe über Daten, d.h. Aktionen a_i kommen von π_β

Bsp. MSE-Loss $\big(V(s_i) - Q(s_i,a_i)\big)^2$ gibt uns Erwartungswert von Q unter π_β (nicht was wir wollen)

Expectile-Loss: $l_2^\tau(x) = \begin{cases} (1-\tau)x^2 & (x > 0) \\ \tau x^2 & (x \leq 0) \end{cases}$

MSE-Loss

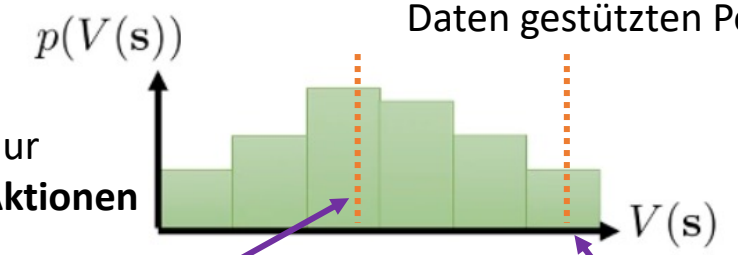




Expectile-Loss

Negative Fehler werden stärker bestraft als positive; keine Overestimation, weil wir $Q(s,a)$ nur auf $(s,a) \in \mathcal{D}$ auswerten

Wert der **besten** durch die Daten gestützten Policy



Verteilung induziert nur durch verschiedene **Aktionen**

$\mathbb{E}_{a \sim \pi_\beta}[Q(s,a)]$ MSE gibt uns das

Expectile-Loss liefert das

Bei $l_2^\tau(x)$ mit τ groß genug:

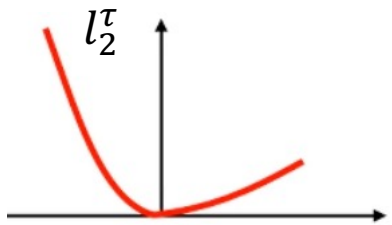
$$V(\mathbf{s}) \leftarrow \max_{\mathbf{a} \in \Omega(\mathbf{s})} Q(\mathbf{s},\mathbf{a})$$

$$\Omega(\mathbf{s}) = \{\mathbf{a} : \pi_\beta(\mathbf{a}|\mathbf{s}) \geq \epsilon\}$$

Implicit Q-Learning (IQL)

"Q-learning with *implicit* policy improvement"

$$Q(s,a) \leftarrow r(s,a) + V(s'), \quad V(s) \leftarrow \mathrm{argmin}_V \frac{1}{N}\sum_{i=1}^{N} l_2^\tau\big(V(s_i,a_i), Q(s_i,a_i)\big)$$



Effektives Verhalten:

Bei $l_2^\tau(x)$ mit τ groß genug:

$$V(\mathbf{s}) \leftarrow \max_{\mathbf{a}\in\Omega(\mathbf{s})} Q(\mathbf{s},\mathbf{a})$$

$$\Omega(\mathbf{s}) = \{\mathbf{a} : \pi_\beta(\mathbf{a}|\mathbf{s}) \geq \epsilon\}$$

$$Q(\mathbf{s},\mathbf{a}) \leftarrow r(\mathbf{s},\mathbf{a}) + \max_{\mathbf{a}'\in\Omega(\mathbf{s}')} Q(\mathbf{s}',\mathbf{a}')$$

"implicit" policy

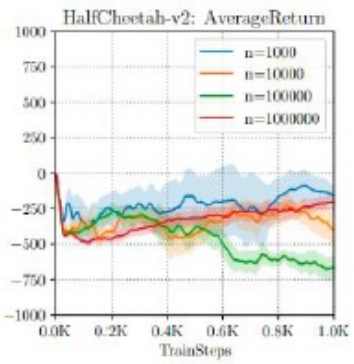
$$\pi_{\text{new}}(\mathbf{a}|\mathbf{s}) = \delta(\mathbf{a} = \arg\max_{\mathbf{a}\in\Omega(\mathbf{s})} Q(\mathbf{s},\mathbf{a}))$$

$Q(s,a)$ Updates können nun gemacht werden ohne out-of-distribution Aktionen zu riskieren, weil nur State-Action-Paare aus dem Datensatz genutzt werden!

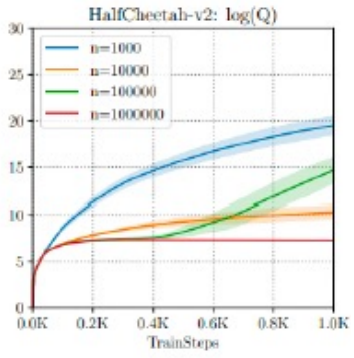
Nach Training von $Q(s,a)$: Policy-Extraktion mit advantage-weighted Methode der vorletzten Folie ($\mathcal{L}_A(\theta)$, mit $A(s,a) = Q(s,a) - V(s)$). (einfach $\mathrm{greedy}(Q)$ funktioniert nicht, da dadurch potentiell Aktionswerte außerhalb der Datenverteilung abgefragt werden würden).

Kostrikov, Nair, Levine. **Offline Reinforcement Learning with Implicit Q-Learning.** '21

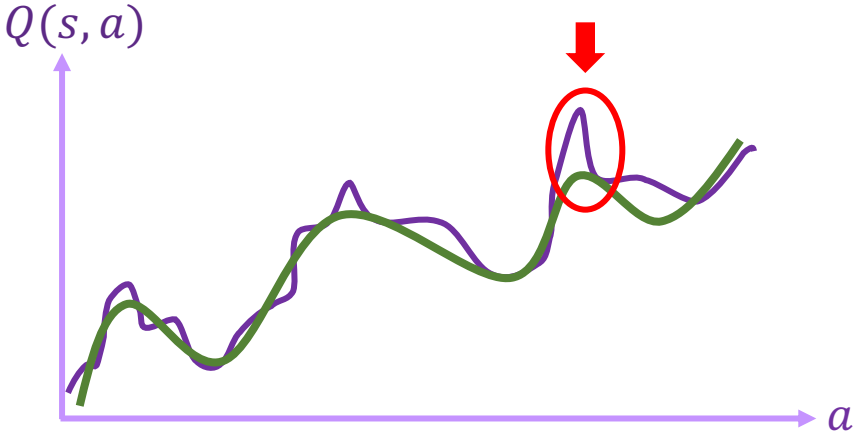
Conservative Q-Learning (CQL)



how well it does



how well it *thinks* it does (Q-values)



Alternativer Ansatz für das out-of-distribution Problem:

$$\hat{Q}^\pi = \operatorname*{argmin}_Q \left\{ \max_\mu \underbrace{\alpha \mathbb{E}_{s\sim\mathcal{D},a\sim\mu(a|s)}[Q(s,a)]}_{\text{Reduziert zu große } Q\text{-Werte}} + \underbrace{\mathbb{E}_{(s,a,s')\sim\mathcal{D}}[(Q(s,a) - (r(s,a) + \mathbb{E}_\pi[Q(s',a')]))^2]}_{\text{Regulärer } Q\text{-Learning Loss}} \right\}$$

Es lässt sich zeigen: $\hat{Q}^\pi \leq Q^\pi$, wenn α groß genug

Conservative Q-Learning (CQL)

Vorige Schranke tendenziell zu pessimistisch – bessere Schranke:

$$\hat{Q}^\pi = \underset{Q}{\operatorname{argmin}} \left\{ \underbrace{\max_{\mu} \alpha \mathbb{E}_{s\sim\mathcal{D}, a\sim\mu(a|s)}[Q(s,a)]}_{\text{Reduziert } Q\text{-Werte } \forall(s,a)} - \underbrace{\alpha \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)]}_{\text{erhöht } Q\text{-Werte, wenn } (s,a)\in\mathcal{D}} \right.$$
$$\left. + \mathbb{E}_{(s,a,s')\sim\mathcal{D}}[(Q(s,a) - (r(s,a) + \mathbb{E}_\pi[Q(s',a')]))^2 \right\} \Bigg\} \mathcal{L}_{CQL}(\hat{Q}^\pi)$$

Der Term $-\alpha \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)]$ hebt die Reduktion der Q-Werte für Daten aus dem Datensatz wieder auf. So werden effektiv nur Q-Werte für Aktionen außerhalb des Datensatzes reduziert.

Es gilt nicht mehr: $\hat{Q}^\pi \leq Q^\pi$, für alle (s,a)

Stattdessen gilt: $\mathbb{E}_{\pi(a|s)}[\hat{Q}^\pi(s,a)] \leq \mathbb{E}_{\pi(a|s)}[Q^\pi(s,a)]$, für alle $s \in \mathcal{D}$

Kumar, Zhou, Tucker, Levine. **Conservative Q-Learning for Offline Reinforcement Learning.** ‘20

Conservative Q-Learning (CQL)

CQL ist ein Actor-Critic Algorithmus, wobei der Critic Q mit dem modifizierten Loss $\mathcal{L}_{CQL}(\hat{Q}^{\pi})$ trainiert wird:

1. Update $\hat{Q}^{\pi}$ durch Minimierung von $\mathcal{L}_{CQL}(\hat{Q}^{\pi})$ und Daten aus $\mathcal{D}$
2. Update Policy π

Bei deterministischer Policy:

$$\pi(a|s) = \mathrm{argmax}_a \hat{Q}(s,a)$$

Bei stochastischer Policy:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \sum_{s \in \mathcal{D}} \mathbb{E}_{a \sim \pi_\theta(a|s)}\left[\hat{Q}(s,a)\right] \quad \text{(Policy Gradient Update)}$$

Conservative Q-Learning (CQL)

Wir brauchen noch einen Regularizer $\mathcal{R}(\mu)$ für μ:

$$\hat{Q}^{\pi} = \operatorname*{argmin}_{Q} \{ \max_{\mu} \alpha \mathbb{E}_{s\sim\mathcal{D},a\sim\mu(a|s)}[Q(s,a)] - \alpha \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)] - \underline{\tau\mathcal{R}(\mu)}$$

$$+\mathbb{E}_{(s,a,s')\sim\mathcal{D}}[(Q(s,a) - (r(s,a) + \mathbb{E}_{\pi}[Q(s',a')]))^2 \}$$

Ohne $\mathcal{R}(\mu)$:

$\max_{\mu} Q(s,a) \rightarrow \max_{a} Q(s,a)$ → nur eine Aktion a wird betrachtet

Q wird nur an dieser Stelle "gefixt"

Typischerweise (insbesondere bei kontinuierlichen Aktionsräumen): $Q(s,a)$ ist für viele Aktionen außerhalb von $\mathcal{D}$ zu groß ist

wir wollen $Q(s,a)$ überall dort verringern → brauchen eine breite Verteilung über alle Aktionen a, für die $Q(s,a)$ groß ist

Conservative Q-Learning (CQL)

Wir brauchen noch einen Regularizer $\mathcal{R}(\mu)$ für μ:

$$\hat{Q}^{\pi} = \operatorname*{argmin}_{Q} \{ \max_{\mu} \alpha \underline{\mathbb{E}_{s\sim\mathcal{D},a\sim\mu(a|s)}[Q(s,a)]} - \alpha \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)] - \underline{\tau\mathcal{R}(\mu)}$$

$$+\mathbb{E}_{(s,a,s')\sim\mathcal{D}}[(Q(s,a) - (r(s,a) + \mathbb{E}_{\pi}[Q(s',a')]))^2 \}$$

Typische Wahl: $\mathcal{R}(\mu) = \mathbb{E}_{s\sim\mathcal{D}}\left[\mathcal{H}\left(\mu(\cdot\,|s)\right)\right]$ maximum entropy regularization

Resultiert in $\mu(a|s) \propto \exp(Q(s,a)/\tau)$

$$\mathbb{E}_{a\sim\mu(a|s)}[Q(s,a)] = \underline{\log \sum_{a} \exp\left(\frac{Q(s,a)}{\tau}\right)}$$

Für diskrete Aktionen: leicht zu berechnen

Für kontinuierliche Aktionen: Importance Sampling aus beliebiger Policy

Offline RL als Sequence Modelling

- Offline-Daten: $(s_1, a_1, r_1, s_2, a_2, r_2, \dots)$
- → Sequenzmodellierung (wie bei Sprachmodellen): Lerne

$$p(\text{next token}|\text{history})$$

- kein Bootstrapping → keine OOD-Q-Fehler
- Decision Transformer (DT) (https://arxiv.org/abs/2106.01345):
  - Input-Sequenz: $(R_1, s_1, a_1, R_2, s_2, a_2, \dots)$ mit $R_t = \sum_{k=t}^{T} r_k$ (Return to go, also ab Zeit t)
  - Loss: $\mathcal{L} = |a_t - \hat{a}_t|^2$, a_t: Dataset-Action, $\hat{a}_t$: Modellvorhersage
  - Inference: Setze Gewünschten Return $R_1 = R^*$
    - Modell-Input: (R^*, s_1), Modell-Output: a_1
    - Environment → (s_2, r_1), Update gewünschter Return: $R_2 = R^* - r_1$
    - Nächster Modell-Input: $(R^*, s_1, a_1, R_2, s_2)$, Modell-Output: a_2
  - "Return-konditioniertes Behavioral Cloning"

Offline RL als Sequence Modelling

- Trajectory Transformer (TT) (https://arxiv.org/abs/2106.02039):
  - Input-Sequenz: $(s_1, a_1, r_1, s_2, a_2, r_2, \dots)$
  - Transformer lernt: $p(\text{next token}|\text{history})$
  - → Generatives Weltmodell
  - Policy wird nicht direkt gelernt → müssen Aktionen sampeln
  - Die Folgen vorhersagen
  - Beste Aktionsfolgen mit Beam Search finden

  "planning"
- Vorteil von Trajectory Transformer:
  - Trajectory Stitching funktioniert (DT kann das meist nicht)
- Vorteil Decision Transformer
  - Einfacher, schneller, kein Beam Search zur Inference-Zeit

Frameworks & Daten

Offline RL Bibliotheken

- d3rlpy
- CORL
- TorchRL (teilweise, Lossfunktionen für CQL, IQL, TD3+BC)

Offline RL Datensätze

- D4RL (klassischer Offline-RL Benchmark)
- Minari, gepflegt von Farama Foundation (→ gymnasium u.v.m.)
  - moderner Dataset-Standard für RL
  - enthält viele portierte D4RL Datensätze
- RL Unplugged
- RoboNet, Bridge (große Robotik-Datensätze mit Daten von echten Robotern)

Typische Domänen

- Locomotion, Manipulation
- Aber auch: Atari, … (“klassisches Online RL”)

Typischer Workflow: Datensatz laden → Algorithmus wählen → offline trainieren → auf Benchmark-Umgebung evaluieren.

Demos

- Siehe `offlinerl.zip` in Moodle (Code & Modell-Checkpoints)